

Accessibility Issue Detection and Repair in Mobile Applications: A Systematic Literature Review

ZHENYU LIU

Beijing Key Lab of Information Service Engineering, Beijing Union University, China, 20231083510901@buu.edu.cn

DENGFENG YAO*

Beijing Key Lab of Information Service Engineering, Beijing Union University, China; Lab of Computational Linguistics, School of Humanities, Tsinghua University, China, tjtdengfeng@buu.edu.cn

HONGZHE LIU

Beijing Key Lab of Information Service Engineering, Beijing Union University, China, liuhongzhe@buu.edu.cn

CHENG XU

Beijing Key Lab of Information Service Engineering, Beijing Union University, China, xucheng@buu.edu.cn

HUIYUE HUANG

Beijing Key Lab of Information Service Engineering, Beijing Union University, China, oasis0207@163.com

Mobile application accessibility is crucial for digital inclusion. This paper presents a systematic literature review (SLR) following PRISMA guidelines, synthesizing advancements in accessibility issue detection and repair techniques. We analyze representative mobile accessibility tools and industrial practices to complement SLR evidence. Nine major databases (ACM DL, IEEE Xplore, ScienceDirect, SpringerLink, Wiley, Web of Science, Scopus, CNKI, arXiv) were searched, with 76 high-quality studies published up to July 2025 analyzed. Common accessibility issues are categorized into four types—perceptibility, operability, understandability, and robustness—aligning with user capability modeling. In accessibility issue detection, evolution from static analysis to deep learning and large language models (LLMs) has shifted research from rule-based matching to semantic reasoning, enhancing automation and generalization. However, detection and repair remain loosely coupled and fragmented, with high false-positive rates and weak inter-module feedback loops. For repair, we propose a taxonomy of rule-driven, learning-based, and LLM-assisted approaches, revealing a gap between detection breadth and repair research depth. LLMs show potential for semantic understanding, issue reasoning, and repair generation, paving the way toward intelligent agents for accessibility. Looking ahead, future research should focus on semantic-enhanced detection, multimodal repair, refined user capability modeling, and unified evaluation standards—collectively advancing accessibility engineering toward an accessibility-by-default design paradigm.

CCS CONCEPTS • Software and its engineering → Software creation and management → Software verification and validation; • Software and its engineering → Software organization and properties → Extra-functional properties → Software usability; • Human-centered computing → Accessibility.

Additional Keywords and Phrases: Accessibility; Mobile Applications; Issue Detection; Automated Repair; Systematic Literature Review; Large Language Models

* corresponding author: Dengfeng Yao.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7392/2026/4-ART

<https://doi.org/10.1145/3809496>

1 INTRODUCTION

In recent years, with the rapid development of mobile Internet and intelligent terminal technologies, mobile applications have been deeply integrated into people's daily lives, becoming core tools for information access, social participation, work completion, and service consumption [1]. According to Ericsson's Mobility Report 2024, by the end of 2024 the number of global mobile subscriptions had reached 8.7 billion, of which smartphones accounted for as high as 82%, totaling 7.16 billion devices [2]. Users can download various mobile apps to access diverse functionalities—such as online shopping, travel navigation, remote work, and health management—thereby enjoying unprecedented convenience brought by digital life [3]. However, while digital services are becoming pervasive, a large number of users still struggle to use mobile apps equally and smoothly due to visual, auditory, motor, or cognitive impairments, facing systematic exclusion caused by the “digital divide.” According to statistics from the World Health Organization, 1.3 billion people worldwide live with significant disabilities involving different degrees of functional impairment, accounting for 16% of the global population [4]. China also has about 85 million people with disabilities and 264 million adults aged 60 and above [5], which means hundreds of millions of users encounter various operational barriers on mobile devices.

Mobile application accessibility refers to the extent to which a mobile software system, throughout its design and implementation, enables users with diverse abilities to perceive content, operate the interface, understand interactions, and complete tasks under robust system support [6]. The Web Content Accessibility Guidelines (WCAG) [7] articulate four fundamental principles—Perceivable, Operable, Understandable, and Robust (POUR)—and, in the mobile context, provide a terminology system and evaluation baseline via mechanisms and assistive technologies (AT)¹ such as screen readers, magnification/captions, target size, focus order, and gestures [8]. In addition, the international standard ISO/IEC 25010 [9] treats accessibility as a sub-characteristic of usability, forming a key component of the software quality model and interlinking with attributes such as compatibility/interoperability, reliability, maintainability, and portability. This offers a standards-based rationale for establishing accessibility as a quality attribute in software engineering (SE) and for systematically operationalizing it across the SE lifecycle (requirements - architecture - implementation - testing - operation and maintenance). Empirical evidence suggests that improvements in accessibility are associated with enhanced user experience [10][11], as well as improvements in overall product quality and market adoption [12][13][14][15]. Accordingly, accessibility in mobile applications has emerged as an important concern in software engineering, with both social and technical implications.

As mobile apps increasingly serve as the primary channel through which the public obtains information, participates in social life, and enjoys services, the existence of standards only lays the conceptual groundwork. To truly realize mobile accessibility, an engineering pathway that is measurable, reproducible, and integrable must be established at the SE level. How to incorporate accessibility from the earliest design stage and sustain it throughout the SE lifecycle [16][17] has thus become an important yardstick for software quality and organizational engineering capability [18][19]. Given platform diversity, device constraints, and interaction complexity, mobile apps pose higher demands on SE practice [20]. Accordingly, both academia and industry have conducted extensive research on accessibility issue detection and repair for mobile apps [21][22][23][24][25][26][27][28] and have achieved progress in automated detection, intelligent repair, and standards-based evaluation. In particular, the introduction of deep learning and LLMs has pushed accessibility detection from rule-based matching toward semantic reasoning, enabling a shift from “surface-level visible defects” to “semantic-level design flaws.” Meanwhile, platform specifications (e.g., accessibility APIs on iOS and Android) and developer tooling (e.g., assistive testing frameworks and automated checkers) continue to mature. This line of work aligns with SE

¹ Assistive Technologies (AT) refer to tools and systems designed to meet the special needs of users with disabilities for perceiving, operating, and understanding information, such as screen readers, screen magnifiers, speech recognition input, subtitle displays, Braille terminals, etc.

's dual emphasis on usability and standardized compliance [29], helping accessibility design evolve from an “ethical advocacy” into an “engineering practice,” and fostering a more inclusive technological ecosystem.

Nevertheless, current research still suffers from fragmented detection capabilities, high false-positive rates, and a weak coupling between detection and repair, and has not yet formed a systematic accessibility assurance mechanism spanning the full SE lifecycle. Despite growing domestic and international efforts, there remains a lack of a systematic and comprehensive survey that synthesizes progress in this field—especially regarding comparative analyses of issue types, detection and repair pathways, system frameworks, and evaluation metrics. Therefore, this paper aims to systematically review research on mobile accessibility detection and repair, and to construct a reproducible evidence baseline and methodological framework, providing the SE community with a verifiable and extensible foundation.

This study conducts a SLR following PRISMA guidelines, covering ACM DL, IEEE Xplore, ScienceDirect, SpringerLink, Wiley, Web of Science, Scopus, CNKI, arXiv, and other databases (time window up to July 1, 2025). The process includes de-duplication based on “title + first author + year (with DOI assistance),” dual-reviewer independent staged screening and quality assessment, as well as backward and forward snowballing. In total, 76 high-quality studies are included.

Based on this corpus, an evidence baseline is established across three main threads—“issues-detection-repair.” An accessibility issue taxonomy is then proposed, aligned with WCAG 2.x (POUR), user abilities, and software engineering concerns, mapping abstract design principles to concrete interaction barriers and repairable elements. The evolution of detection techniques is further analyzed along a trajectory from “static → dynamic → hybrid → model-driven → LLM-assisted,” with a comparative examination of their trade-offs in coverage, automation level, false-positive cost, and platform adaptation. Finally, repair strategies in mobile contexts are systematically synthesized, including rule-driven/optimization-based, machine learning-based, and LLM-assisted approaches. Their assumptions, effectiveness, and potential side effects are analyzed, revealing key breakpoints in the “detection-repair” chain and possible bridging directions.

These findings provide actionable insights for the development of high-quality, accessibility-oriented software engineering systems.

- (1) We conduct a PRISMA-guided SLR, consolidating mobile accessibility research up to July 1, 2025, and, after de-duplication, screening, and quality assessment, include 76 high-quality papers as a starting point for future work on accessibility issue taxonomy, detection, and repair.
- (2) We propose an accessibility issue classification framework compatible with WCAG, user abilities, and SE perspectives, mapping abstract accessibility principles to specific interaction barriers, thereby providing theoretical support for issue modeling and method selection.
- (3) We distill a five-stage evolution pathway for detection techniques—“static-dynamic-hybrid-model-driven-LLM-assisted”—and systematically analyze their similarities and differences in terms of coverage, automation, and platform adaptation.
- (4) We complement the underexplored “repair” dimension of mobile accessibility by summarizing mainstream repair strategies, discussing their limitations, and exploring the application potential of emerging techniques such as LLMs for automated repair.
- (5) By integrating accessibility standards and regulations worldwide, we discuss challenges and prospects for evaluation-system construction and tooling-level engineering deployment, offering references for building verifiable, transferable, and continuously evolving accessibility-oriented SE frameworks.

The remainder of this paper is organized as follows. Section 2 reviews background and related work; Section 3 describes the survey methodology. Sections 4-6 establish an accessibility issue taxonomy for mobile apps and map it to WCAG (Section 4), summarize detection methods across stages and analyze their assumptions, inputs/outputs, coverage, and false-positive costs (Section 5), and synthesize rule/optimization-based, learning-based, and LLM-assisted repair strategies with a discussion of their background, approaches, scope, strengths, and weaknesses (Section 6). Section 7 discusses challenges and future directions; Section 8 outlines threats to validity and limitations; and Section 9 concludes the paper.

2 BACKGROUND AND RELATED WORK

2.1 Accessibility Standards & Guidelines

As a critical quality attribute, mobile application accessibility must rely on a clear and operational standards system to be effectively implemented. On the one hand, accessibility standards provide unified technical specifications for development, design, and testing, ensuring the systematic integration of accessibility throughout the software lifecycle; on the other hand, they are prerequisites for building automated detection tools and enabling quantitative evaluation. At the same time, the standards system also underpins the formulation and supervision of accessibility policies. Therefore, establishing scientific, authoritative, and continuously evolving accessibility standards is a key foundation for promoting fairness and accessibility in digital environments.

At present, a relatively comprehensive global standards system for accessibility has been established. Table 1 summarizes representative standards and guidelines, ranging from internationally applicable rules (e.g., WCAG² 2.x[5][31], EN 301 549), to mainstream platform specifications (e.g., Android Accessibility[32], iOS Human Interface Guidelines[33]), and further to national-level policy standards (e.g., GB/T 37668-2019 General Technical Requirements for Accessibility Software [34]). Together these form a multi-layer support system from content principles to platform adaptation. These standards specify systematic technical requirements in terms of perceivability, operability, understandability, and robustness, offering broad design references for user groups with different abilities. In particular, at the mobile-platform level, both iOS and Android integrate rich AT APIs (e.g., VoiceOver, TalkBack, accessibility labels, focus management), significantly reducing the cost of implementing accessibility features. Moreover, the “four principles” proposed by WCAG are highly abstract and generalizable, and have been widely used to guide accessibility design for web content and parts of mobile app semantic structures.

Table 1: Summary of Main Standards for Mobile Application Accessibility

Standard Name	Issuing Organization	Scope of Application	Main Content	Features & Limitations
Web Content Accessibility Guidelines (WCAG 2.1) ³	W3C released June 5, 2018	Web, some mobile apps	principles: Perceivable, Operable, Understandable, Robust	Mature, international; web - centric, weak on mobile - specific features (touch/voice)
Mobile Accessibility Best Practice ⁴	W3C/WAI released Feb 26, 2015	Mobile Web, Apps	WCAG application in mobile scenarios	Focuses on mobile features (rotation, small screens); still web - semantic - based
EN 301 549 Accessibility Requirements for ICT Products/Services ⁵	CEN, CENELEC, ETSI	All ICT products/services (websites, apps, etc.)	Unifies accessibility clauses; integrates WCAG for compliance checks	Comprehensive, EU - aligned; high technical barriers, EU - focused
Android Accessibility Dev Guidelines ⁶	Google	Android mobile apps	TalkBack support, semantic labeling, gestures	Full API/tools; high dev threshold, reliant on system updates

² WCAG 2.0 [7] provides almost no guidance on creating accessible content for mobile devices. To fill this gap, W3C released MWBP[30]. MWBP is a rule-defining document intended to make websites more usable on mobile devices. Similar to WCAG, MWBP defines a set of checkpoints to ensure online content is suitable for access from mobile devices. Shortly after MWBP was published, W3C also released an update to WCAG: WCAG 2.1.

³ <https://www.w3.org/TR/WCAG21/>

⁴ <https://www.w3.org/TR/mobile-bp/>

⁵ <https://www.etsi.org/human-factors-accessibility/en-301-549-v3-the-harmonized-european-standard-for-ict-accessibility>

⁶ <https://developer.android.com/guide/topics/ui/accessibility>

Standard Name	Issuing Organization	Scope of Application	Main Content	Features & Limitations
iOS Accessibility Dev Guidelines ⁷	Apple	iOS mobile apps	VoiceOver, UIAccessibility protocol	Well - integrated, mature ecosystem; closed - source limits tool innovation
GB/T 37668 - 2019 General Technical Requirements for Accessible Software ⁸	Standardization Administration of China	All software (including mobile apps)	Functionality, visual, auditory, interaction requirements	Legally binding, localized; general rules, lacks depth
Huawei Accessibility Dev Guidelines ⁹	Huawei Dev Alliance	HarmonyOS, EMUI - based Android devices	Text labels, touch focus, voice, color contrast, accessibility tools	Integrates with HarmonyOS, supports multi - modal interaction; few auto - test tools
MIUI Accessibility Dev Manual ¹⁰	Xiaomi Dev Center	MIUI - based Android apps/services	TalkBack compatibility, auxiliary services, touch/text specs	Practical for Android devs; reliant on Android native capabilities

Compared with desktop apps or web content, mobile application accessibility involves more touch, speech, and multimodal interaction features, and thus requires targeted design based on platform-specific development tools and testing mechanisms. For example, Apple iOS provides the VoiceOver speech feedback system and the Accessibility Inspector [35] visualization tool, while Android integrates the TalkBack screen-reading service and the Accessibility Scanner [36] automated checker. In recent years, Chinese vendors such as Huawei and Xiaomi have also released their own accessibility guidelines and capability APIs, further improving native platform support.

Although these standards have played a positive role in advancing accessibility, many objective problems and technical challenges remain in practice and deserve attention:

- 1) Insufficient applicability. Current accessibility evaluation still mainly relies on Web-oriented guidelines such as WCAG, lacking standards dedicated to mobile contexts. Matos et al. [37] point out that mobile accessibility evaluation lacks mobile-specific guidelines, forcing evaluators to refer to documents originally intended for web pages. As a result, there are no explicit requirements for mobile-specific interactions such as multi-touch gestures, sensor-based input, or voice commands, making it difficult for existing standards to effectively cover accessibility issues in these functionalities.
- 2) Poor cross-platform compatibility. Although native Android and iOS each provide accessibility APIs, they are inconsistent, and cross-platform development frameworks do not fully expose these differentiated capabilities. Mascetti et al. [38] found that while iOS and Android screen-reader APIs overlap substantially in functionality, mainstream cross-platform frameworks (e.g., React Native, Xamarin) often encapsulate only part of them, with some basic accessibility interfaces not supported at all. If developers need such missing functions, they must write platform-specific code, which increases development and testing complexity and requires separate accessibility adaptation for different platforms.
- 3) Insufficient technical granularity of national standards. China's GB/T 37668-2019, as the first domestic technical standard for accessibility software, carries positive institutional significance, but provides limited fine-grained guidance for real operational scenarios. It largely follows WCAG and related standards to regulate accessibility for web pages and mobile websites/apps, and does not offer specific guidance for emerging interaction forms such as wearables, VR/AR, or AI (W3C only released the XR Accessibility User Requirements (XAUR) guideline in 2021 [39]). International studies [40] show that for these new-technology domains, accessibility recommendations often lack unified standards comparable to WCAG.

⁷ <https://developer.apple.com/design/human-interface-guidelines/accessibility>

⁸ <https://openstd.samr.gov.cn/bzgk/gb/newGbInfo?hcno=35ECC696805C1A67C93B74FB6D0D8EFB>

⁹ <https://developer.huawei.com/consumer/en/doc/harmonyos-guides/accessibility-overview>

¹⁰ <https://dev.mi.com/xiaomihyperos/documentation/detail?pld=2031>

4) Weak dynamic adaptation. Existing standards usually adopt static checking and lack mechanisms for dynamically adjusting interfaces to individual user needs. The literature indicates [41] that user modeling can generate and dynamically adapt interfaces to accommodate diverse preferences and needs. However, because standards do not support interoperability of user models, mobile apps cannot optimize interfaces in real time based on differences in language, interaction habits, or disability types, thus neglecting personalized requirements [42].

5) Lack of interdisciplinary theoretical support. Mobile application accessibility is not only a matter of technical standards, but also a multidisciplinary systems endeavor involving ergonomics, SE, cognitive science, and HCI. A study on design guidelines for users with multiple disabilities found [43] that experts widely recognize the need for cross-disciplinary involvement in guideline development. Yet current standards lack support mechanisms to systematically integrate such interdisciplinary theories into accessibility design, weakening the rigor and scientific grounding of the methodology.

Therefore, the WCAG 2.x-centered standards system has provided clear goals and principles for mobile apps, and, together with EN 301 549 and platform development guidelines (Android/iOS, etc.), constitutes a reliable baseline [44]. The key challenges now lie in how to refine abstract clauses into executable control-/state-level checks and repair constraints, and how to implement and verify them in CI/CD pipelines. Accordingly, we take WCAG as the benchmark to map guideline clauses to concrete issue types, while specifying the evidence items required for determination, thereby offering a unified reference for subsequent studies (e.g., dataset construction, benchmark evaluation, and cross-tool alignment).

2.2 Mobile Application Accessibility Detection

Mobile accessibility detection aims to automatically or semi-automatically identify whether interfaces and interactions violate WCAG and platform guidelines. It plays a crucial role in supporting developers to incorporate accessibility design throughout the entire software lifecycle, and meanwhile provides technical assurance for systematic accessibility evaluation [45]. Common evidence sources include: the accessibility tree (e.g., value/state, focus order), layouts/resources (XML), pixel-level screenshots (contrast, touch target size, occlusion), and runtime events (TalkBack/VoiceOver callbacks).

In recent years, both industry and academia have made substantial progress in mobile accessibility detection. A variety of practically valuable tools and frameworks have emerged, and detection methods and technical routes have shown increasingly diversified and intelligent development trends.

At an early stage, mobile accessibility evaluation mainly relied on manual testing: accessibility experts or real users, under AT conditions, simulated real interactions to assess the accessibility properties and structural design of UI elements [46]. Such methods can identify design defects that hinder understanding and operation in depth, but they heavily depend on human judgment, incur high labor costs, and exhibit poor reproducibility; in fast-iterating development processes, it is difficult to balance coverage and timeliness [45][47]. Especially when mobile app versions are updated frequently, the maintainability of manual evaluation declines markedly, limiting its long-term usability. To address these challenges, research and engineering practice gradually shifted toward automated detection to improve efficiency, reproducibility, and issue localization capability [48]. Automated approaches typically embed accessibility checks into development frameworks or testing workflows, enabling developers to perform accessibility detection alongside functional testing and thus realizing an integrated “testing-as-detection” paradigm. Following the traditional view, these approaches are often categorized into manual inspection, automated detection, and user testing [49]. As methodologies evolved, the research community re-examined detection mechanisms along two axes: (1) the analysis-strategy axis—static analysis (source code, UI layouts, and resources) versus dynamic analysis (runtime UI states and interaction behaviors) [23][50]; and (2) the development-process axis—development-time detection versus runtime detection [51][52]. The former is suitable for early testing during the build stage, whereas the latter emphasizes simulating real user usage scenarios.

At the survey level, besides taxonomy-based studies, some work focuses on specific detection pathways and tool ecosystems. Silva et al. [45] reviewed mobile automated evaluation tools, categorizing them into static/dynamic/crawling approaches and comparing mainstream frameworks; they pointed out insufficient automated coverage and reproducibility issues, but the work is relatively early and does not address learning/LLMs or pixel-level methods. Matos et al. [37] organized evaluation methodologies, highlighting inconsistencies and tool scarcity caused by the “semantic backporting” of web guidelines to mobile, yet they do not compare concrete detection algorithms nor their repair integration. Acosta-Vargas et al. [44] provided a scoping review of native mobile accessibility, covering disability types, guidelines, and evaluation practices, but with limited technical granularity. Seixas Pereira et al. [49] discussed methodological challenges from the angle of early evaluation and workflow embedding. Nie et al. [28], from a cross-platform perspective, examined issues, tools, and datasets, clarifying that most detectors suffer from limited coverage and dataset bias; however, they mainly list findings without a systematic issue-organizing logic or technical descriptions of detection-repair pipelines. Yu et al. [26] summarized visually driven UI modeling, widget detection, and exploration strategies that support accessibility crawlers, but they do not focus on accessibility issue types or repair. A survey on ML-based automated testing for Android [53] outlined trends from static/dynamic to vision/RL/LLMs, yet it does not follow accessibility standards.

By jointly considering detection mechanisms, analysis strategies, and levels of intelligence, we propose a five-category taxonomy: static detection, manual dynamic inspection, dynamic testing framework integration, GUI automated exploration and behavior analysis, and AT simulation with intelligent-augmented detection. This taxonomy is intended to more systematically present current mainstream methods and their evolutionary trajectory. Compared with prior classifications centered on detection phases or analysis strategies, our taxonomy better reveals differences among methods in detection capability, applicable scenarios, and integration depth, providing a more actionable framework for subsequent research and systematic tool development.

2.3 Mobile Application Accessibility Repair

Accessibility repair focuses on translating “detected issues” into implementable interface/code changes. It aims to preserve accessibility while minimizing disruption to original visuals and interactions. Compared with detection, repair has a narrower reachable scope, because many strategies depend on prerequisites (e.g., adjustable style parameters, available text resources, and no mobile app crashes after modification). With the continuous improvement of mobile accessibility detection capabilities, the research focus has gradually shifted from merely identifying problems to proposing effective repair strategies. This shift is not a simple technical iteration, but an inevitable trend driven by real development needs. Although detection methods can uncover a large number of accessibility issues [23][54][55][56][57][58][59], leaving repair entirely to developers often leads to efficiency bottlenecks and a lack of specialized expertise. This has motivated ongoing research toward automated repair, so as to reduce developer burden and improve repair efficiency.

Despite many explorations in accessibility repair, systematic surveys in this area remain scarce. Nie et al. [28] offer mainly an overview: although they touch on issues and tools, their organization of methodological mechanisms and comparative dimensions is still not systematic, making it difficult to compare, accumulate, and reuse methods across studies. To fill this gap, we, for the first time, propose a repair-technique taxonomy for mobile apps from two dimensions: (1) the degree of automation, and (2) the way repair is embedded into the development workflow. We summarize three mainstream categories: rule-driven and optimization-oriented repair, ML-based repair, and LLM-assisted repair. This taxonomy helps clarify the technical routes, engineering feasibility, and evolutionary trends of different methods, providing both theoretical grounding and practical references for future research and system design.

3 SURVEY METHODOLOGY

In this survey, we adopt the SLR paradigm. Specifically, we formulate research questions (RQs), design search strings and data sources, define inclusion/exclusion criteria, conduct independent manual screening, perform quality assessment (QA), and finally synthesize and analyze the collected evidence.

3.1 Research Questions

Our objective is to summarize, categorize, and analyze published studies related to accessibility issues, accessibility detection, and accessibility repair. Centered on issue taxonomy, detection, and repair for mobile application accessibility, and aligned with the four WCAG 2.x principles, we address the following three RQs:

RQ1. How can mobile application accessibility issues be systematically classified and mapped under the WCAG framework? What are the manifestations and triggering contexts of each category in mobile settings, and how do they relate to user abilities?

RQ2. What is the evolutionary trajectory of mobile application accessibility detection techniques, and what coverage, automation level, strengths/limitations, and applicable scenarios does each line of work exhibit?

RQ3. How can the landscape of mobile application accessibility repair techniques be categorized, and what are their repair pipelines, generalizability, and functional performance?

By answering these RQs, we are able to: (i) unify terminology and granularity across studies, creating a shared issue space for subsequent benchmarking and comparison; (ii) trace the evolution of detection methods and provide comparable analytical dimensions in terms of coverage, automation level, strengths/limitations, and applicable scenarios, thereby supporting method selection and tool reuse; and (iii) construct a strategy-oriented synthesis for repair, distilling the use cases, pros/cons, and feasibility gains and limitations of LLMs in repair, helping both researchers and practitioners identify which types of issues are best addressed by which repair approaches.

3.2 Search Strategy and Paper Collection

To collect relevant literature as accurately and comprehensively as possible, we iteratively discussed the protocol in multiple meetings, consulted peer experts, referred to related SLR guidelines [60], and cross-checked several existing surveys. We organized our search and screening following the workflow below (credibility tiering, initial search, terminology expansion, batch automated search, manual screening, snowballing, and QA). The overall workflow is shown in Fig. 1.

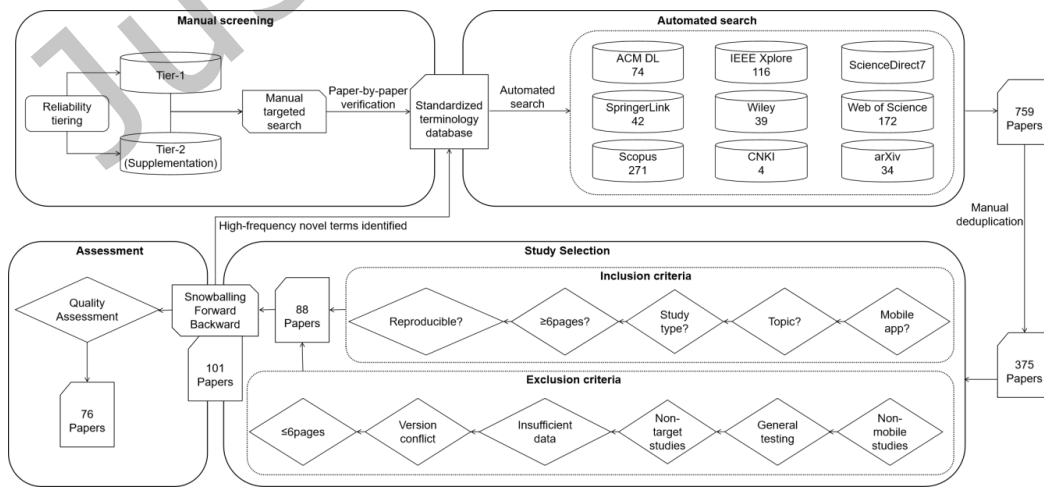


Figure 1: General workflow of the paper collection

Step 1 | Define the search scope and tier venues by credibility.

We tier target venues by credibility. Tier-1 (high-trust) includes ICSE, ESEC/FSE, ASE, ISSTA, ICSME, SANER, CHI, ASSETS, UIST, MobileHCI, TACCESS, TOSEM, TSE, EMSE, IST, and JSS. Tier-2 (supplementary) includes PMC, IJHCI, high-quality regional conferences, and authoritative Chinese journals. In addition, arXiv was searched as a supplementary source to improve coverage and capture emerging terminology and frontier work in this rapidly evolving area. The time window ends on July 1, 2025.

Step 2 | Conduct an initial search with basic hub keywords.

Guided by the RQs, we first performed a round of targeted manual search within Tier-1 venues using low-barrier hub terms to quickly locate directions and representative works on mobile accessibility detection/repair. English queries include (“accessibility” OR “wcag” OR “disability”) AND (“mobile app” OR “Android” OR “iOS”) AND (“repair” OR “detection”). Chinese queries include (“无障碍” OR “可及性” OR “可达性” OR “可访问性”) AND (“移动” OR “安卓”) AND (“检测” OR “修复”).

Step 3 | Build a standardized terminology database.

We examined titles/abstracts/keywords paper by paper to form an initial seed set, ensuring coverage of all three threads (issues-detection-repair). We then extracted terms and expanded synonyms (case variants, abbreviations, platform-name unification) from titles/abstracts/introductions/threats-to-validity/conclusions, producing a standardized terminology database. Based on this database, we conducted a second round of manual supplementary search in Tier-1 venues until term coverage saturated. During screening, we removed terms such as “wcag” and “disability” and added “software,” “issue,” “testing,” “evaluation,” and “GUI.” This is because our search results showed that “wcag” is mostly used for Web and generic guidelines, causing excessive hits on Web-accessibility literature, while “disability” leads to many social-science/medical papers, producing high noise. Through close reading of qualifying papers, we found that detection-related terms are not limited to “detection” but also include “testing” and “evaluation,” and similar expansions apply to terms like “software,” “issue,” and “GUI.”

After consolidation, the final term clusters we used are:

English queries: (“software” OR “app” OR “mobile” OR “Android” OR “iOS”) AND (“issue” OR “testing” OR “detection” OR “evaluation” OR “assessment” OR “repair”) AND (“accessibility” OR “GUI”).

Chinese queries: (“无障碍” OR “可及性” OR “可达性” OR “可访问性”) AND (“移动应用” OR “移动端” OR “安卓”) AND (“检测” OR “修复” OR “评估” OR “测试”).

Step 4 | Run batch automated searches across multiple databases.

To achieve high coverage, low duplication, and reproducibility, we executed batch searches in ACM DL, IEEE Xplore, ScienceDirect, SpringerLink, Wiley, Web of Science, Scopus, CNKI, and arXiv (ACM/IEEE as core hosting sources; Scopus/WoS for cross-publisher de-duplication and citation tracking; publisher platforms for journal completion; CNKI for Chinese literature; and arXiv as a supplementary source for emerging work and terminology). Together, these databases cover major publications in SE, usability, and HCI, and thus reflect both core results and frontier trends of mobile accessibility research. A preliminary comparison indicated that their coverage already subsumed relevant content indexed in Engineering Village and similar engineering databases.

For each arXiv record, we first checked whether a peer-reviewed version existed. If a formal peer-reviewed version was available, we retained the formal version and treated the arXiv version as a duplicate/version conflict. If no peer-reviewed version was available, the arXiv paper was not excluded solely for being a preprint; instead, it proceeded to the same relevance screening and QA process as the

other records. Only arXiv-only papers that demonstrated sufficient relevance and quality were finally retained. We then merged all results and de-duplicated by “title + first author + year” (with DOI-assisted matching). This process retrieved 759 papers in total: 74 from ACM DL, 116 from IEEE Xplore, 7 from ScienceDirect, 42 from SpringerLink, 39 from Wiley, 172 from Web of Science, 271 from Scopus, 4 from CNKI, and 34 from arXiv. After de-duplication, 375 papers remained.

Step 5 | Define inclusion/exclusion criteria and conduct manual screening.

After batch retrieval, we evaluated relevance according to predefined inclusion and exclusion criteria (Tables 2 and 3) to ensure that the selected papers directly addressed our RQs. After this step, 88 papers remained. It should be noted that, because page policies vary across venues, we did not use a fixed page-length threshold as the sole criterion for identifying full papers. Instead, we relied primarily on the formal publication type and the completeness of the reported methodology, data, and results. Page count was used only as a supporting cue to distinguish full research papers from posters, demos, tutorials, editorials, and other non-formal publication types.

Table 2: Inclusion criteria

ID	Criterion	Specific decision points.
①	Platform scope	The study explicitly targets mobile apps (data/experiments/evaluations conducted on Android/iOS apps).
②	Accessibility focus	The topic is explicitly Accessibility, mappable to WCAG 2.x, or clearly involves users with disabilities/AT.
③	Study type	Falls into one of three types: (i) user analysis/interviews; (ii) large-scale empirical studies (market/codebase/benchmark results, etc.); (iii) new techniques (accessibility detection/repair methods/systems/tools).
④	Publication form	Peer-reviewed journal/conference full papers, as well as SLRs and exceptionally retained arXiv-only papers that satisfied the same inclusion, exclusion, and QA criteria
⑤	Traceability/reproducibility	Public implementations/configurations/sample data/material links, or descriptions sufficient to support reproduction.

Table 3: Exclusion criteria

ID	Criterion	Specific decision points.
①	Non-mobile	Web/desktop-only studies; or non-UI accessibility topics such as “urban/building/transport/network accessibility.”
②	Generic GUI defects / marginal populations	General usability/performance/energy/security/privacy/ads, or “general usability-only” studies without disability groups or alignment to WCAG/AT contexts.
③	Non-target study type	Not belonging to the three target types: user analysis/interviews; large-scale empirical studies; new detection/repair techniques/systems/tools.
④	Insufficient data	Full text unavailable; or full text cannot confirm platform/type/accessibility relevance.
⑤	Duplicates/version conflicts	Duplicate publications/extended versions; coexistence of preprint and formal version.
⑥	Ineligible length/form	Posters/short papers/tutorials/editorials/news/demos and other non-formal research papers.

Step 6 | Snowballing.

We performed backward (references) and forward (citations/latest on-topic) snowballing on the included papers. Newly found items were first screened with Step 5 criteria. We set up terminology sentinels: once high-frequency new terms appeared in snowballing, we immediately returned to Step 3 to add them to the terminology database and reran batch search plus manual checking to avoid omissions. After snowballing, 101 papers were collected.

Step 7 | Quality Assessment (QA).

While ensuring coverage, we controlled literature quality and bias risk to prevent off-topic or low-quality studies from entering the sample, thereby improving credibility, auditability, and generalizability. Following classic references [61][62][63][64], we evaluated papers using an adapted QA rubric with five items:

- QA1. Precision of research objectives and scope.
- QA2. Clarity of research methods and design.
- QA3. Transparency and sufficiency of data/sample sources.
- QA4. Soundness of evaluation and analysis metrics.
- QA5. Reproducibility of results.

For QA1, 0 means objectives/targets are vague and not focused on mobile application accessibility; 1 means mobility or accessibility is involved but with unclear boundaries and insufficient WCAG alignment; 2 means the study clearly focuses on mobile accessibility, frames measurable RQs, and aligns explicitly with WCAG or mainstream AT behaviors.

For QA2, 0 means the study design/workflow is unclear with missing key variables/comparisons/experimental setups; 1 means a basic workflow is given but lacks details; 2 means the method-data-results chain is coherent, with sufficient descriptions of platform/devices, variables and processing, comparisons and control strategies.

For QA3, 0 means data/sample sources are unknown or clearly insufficient, without inclusion/cleaning criteria; 1 means sources and scale are provided but incomplete, with vague cleaning/QC; 2 means sources, scale, inclusion/exclusion, and cleaning are transparent, with mobile app lists and justified scale/representativeness.

For QA4, 0 means no appropriate metrics or only descriptive outcomes; 1 means metrics exist but definitions are inconsistent, lacking fair baselines or statistical tests/ablations; 2 means metrics match tasks (e.g., detection: Precision/Recall/F1/coverage/false positives; repair: success rate/introduced issues; user studies: task success/effectiveness/satisfaction) and are supported by fair baselines, ablations/significance tests, or confidence intervals.

For QA5, 0 means no reproducible materials or key implementation details; 1 means partial code/data/configurations are available but documentation/versioning is insufficient; 2 means accessible code, data/lists, scripts, and configurations are complete, with environments/dependencies specified so major results can be audited.

After passing inclusion/exclusion screening, each paper was scored on the five QA items using a 0/1/2 scale (total 10 points). Papers scoring ≥ 6 were considered acceptable. Two reviewers scored independently and computed agreement; disagreements were resolved through discussion. After QA, 76 papers were finally retained. This sample size is comparable to prior systematic literature reviews in software engineering and HCI domains, ensuring sufficient coverage and representativeness of the research landscape.

3.3 Publication Statistics

After paper collection, we conducted descriptive statistics and examined emerging publication trends. Fig. 2(A) summarizes the venue distribution of the finally included studies. Of the 76 included papers, 70 (92.1%) were published in peer-reviewed journals or conference proceedings, while the remaining 6 papers (7.9%) were arXiv-only studies. Among the 34 records retrieved from arXiv, these 6 arXiv-only papers were finally retained in the 76-paper sample because no peer-reviewed version was available and they satisfied the same inclusion, exclusion, and QA criteria as the rest of the sample; the remaining arXiv records were either represented by corresponding peer-reviewed versions or excluded during screening and QA. Among the formally published venues, CHI is the most frequent venue (18.4%), followed by ICSE (6.6%) and ASE (6.6%), then ESEC/FSE (5.3%) and TSE (5.3%). Other representative venues include TACCESS (3.9%), ICSME/ASSETS/TOSEM (2.6% each), and SANER/EMSE/IST/JSS (1.3% each), while “Other” accounts for 31.6%. Overall, this distribution indicates that mobile accessibility detection and repair has attracted sustained attention from both the software engineering and HCI communities.

The yearly distribution is shown in Fig. 2(B). Since 2022, publication volume has increased notably: only 3 papers in 2019; 7 papers each in 2020 and 2021; then a substantial growth with 15 papers in 2022, 13 in 2023, 17 in 2024, and 8 papers published by July 1, 2025. This trend indicates sustained and rising academic attention to mobile accessibility. As more preprints enter peer review and formal publication, outputs are expected to continue growing in the foreseeable future.

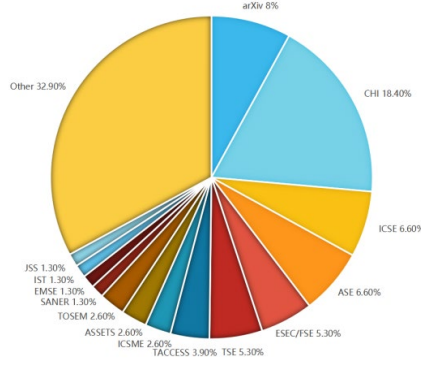


Fig. 2(A) Distribution of papers by venue

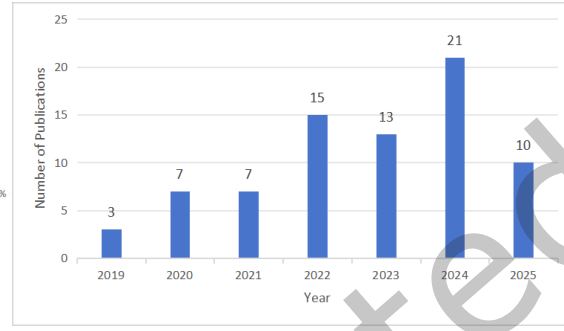


Fig. 2(B) Number of publications per year

As shown in Fig. 3(A), 70% of the papers are published in conference proceedings, while 30% appear in journals. Fig. 3(B) further shows that, among the 76 studies, 8 are related to accessibility guidelines/standards, 22 are large-scale empirical studies, 9 are user analysis/interview studies, 20 fall into accessibility detection techniques, and 17 fall into accessibility repair techniques.

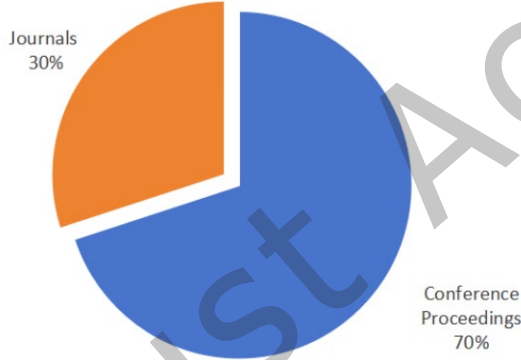


Fig. 3(A) Publication venue types

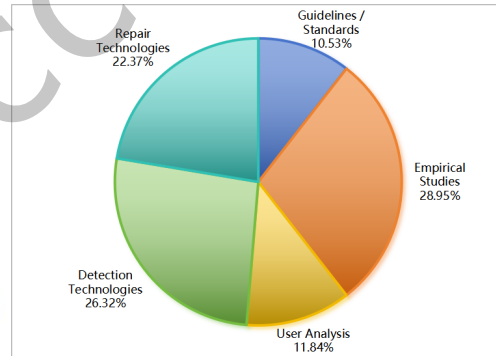


Fig. 3(B) Contribution types

4 HOW CAN MOBILE APPLICATION ACCESSIBILITY ISSUES BE SYSTEMATICALLY CLASSIFIED AND MAPPED UNDER THE WCAG FRAMEWORK? WHAT ARE THE MANIFESTATIONS AND TRIGGERING CONTEXTS OF EACH CATEGORY IN MOBILE SETTINGS, AND HOW DO THEY RELATE TO USER ABILITIES?

In this section, grounded in WCAG's four principles, we build a mapping scheme from issue types to user-ability models and to engineering capability domains. This mapping is intended to align with SE: it provides observable anchors for detection and repair at each stage, so that accessibility shifts from "late-stage remediation" toward "end-to-end lifecycle embedding."

4.1 From Standards to Engineering: Redefining the Relationship Between Accessibility and the Software Lifecycle

Accessibility is not a local optimization at the UI layer; rather, it is a systemic engineering requirement driven by differences in user abilities[65]. Users with visual impairments rely on voice navigation and linear focus paths[66]; users with hearing impairments rely on captions and visual cues[67]; users with cognitive impairments and older adults rely on consistent[68], low-burden interaction structures. Therefore, systems must support diversified perception and operation pathways, rather than merely satisfying interface-level traits such as “visible” and “clickable.” This requirement is especially salient in mobile app contexts characterized by touch-centric interaction, limited viewport, and strong reliance on mobile AT.

However, the long-standing lag of accessibility to the end of development is not caused by a lack of technology, but by process discontinuities. Studies show only 37% of teams incorporate accessibility requirements at the requirements stage, which increases rework costs by 60%; 58% of teams do not use accessibility guidelines during design, leading to prototype defect rates as high as 75% [19]. As a result, late-stage “patch-style repair” is not only costly and ineffective, but also fails to produce a consistent user experience. The root cause is that WCAG and ISO define targets such as perceivability, operability, understandability, and robustness (the What), but do not specify how to implement them, who is responsible, or at which stage to enforce them (the How). The absence of a responsibility mapping from standards goals to engineering mechanisms leads to accessibility being misconstrued as a UI add-on rather than a manageable software quality attribute.

To close this gap, we propose a “framework for mapping accessibility standards to engineering practice” (Fig. 4). The framework follows a Why-What-How logic: user-ability differences constitute the source of requirements (Why); WCAG/ISO are abstracted into standards goals (What); these goals are further mapped into executable engineering capability domains (How), including information perception, interface operation, interaction logic, semantic structure, multimodal support, API compatibility, event management, and system stability. Through a one-way mapping, abstract principles are transformed into implementable, verifiable, and maintainable engineering responsibility units. This framework strictly separates standards goals from engineering mechanisms: the four principles are no longer treated as system capability types, but as normative constraints over engineering capability domains. The capability domains themselves are measurable, assignable, and integrable, providing structural carriers for detection, repair, and automated assurance. They are also consistent with the “perception-action-cognition” model, offering cognitive plausibility rather than an experience-based taxonomy.

The formation of engineering capability relies on four collaborative mechanisms: (1) user-ability differences determine system support boundaries; (2) AT provide alternative input/output channels (e.g., Accessibility Service); (3) the mobile app layer exposes interpretable semantic structures and focus orders; and (4) the system layer guarantees cross-device robustness and event pathways. This shows that accessibility is a cross-layer collaborative outcome rather than a single UI attribute.

Accordingly, Fig. 4 provides an executable path from normative principles to engineering mechanisms, redefining accessibility as a software quality attribute that is driven by ability differences, constrained by standards goals, and realized through engineering capabilities. Accessibility is no longer a supplementary requirement added at the end of development, but a systemic objective that must be designed, implemented, verified, and maintained.

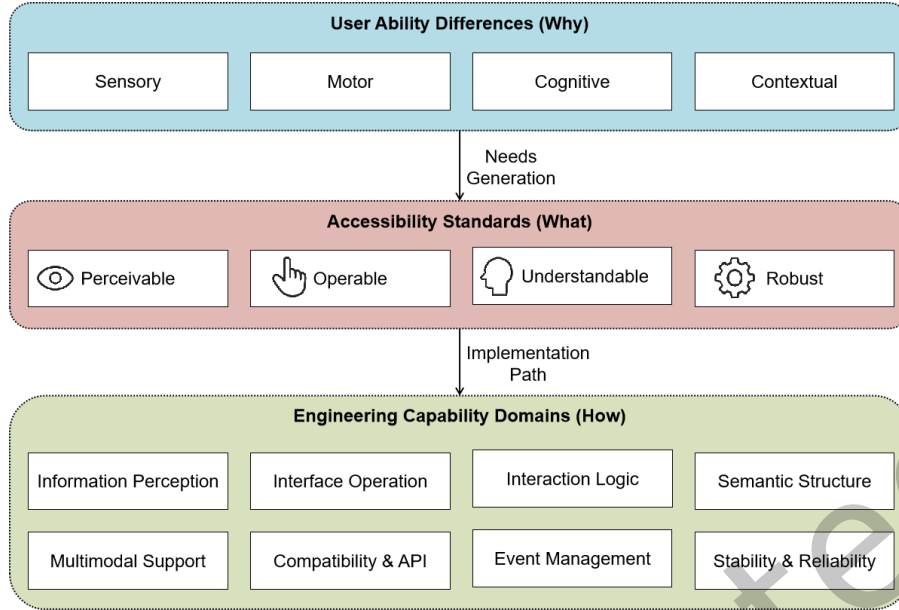


Figure 4: Framework for Mapping Accessibility Standards to Engineering Practice

The framework translates accessibility standards into engineering capability domains that can be embedded into the software development lifecycle, providing a practical route from principles to implementation responsibilities.

Although the framework is cross-platform in nature, its engineering path is particularly suited to mobile app contexts, where the ecosystem faces device fragmentation, touch-first operation modes, and heavy dependence on mobile AT (e.g., TalkBack, VoiceOver). Using the framework only requires identifying user-ability needs and partitioning engineering responsibilities according to standards goals, without reliance on specific tools or platforms. It supports requirements modeling, design specifications, usability evaluation, and AT-compatibility validation. The expected benefits manifest at three levels: engineering-wise, shifting from end-stage patching to early-stage prevention; user-wise, improving information accessibility and task efficiency; and system-wise, forming transferable and reproducible structural baselines that enable continuous evaluation and maintenance. Thus, the framework not only provides structural foundations for engineering accessibility issues on mobile, but also offers a pathway for mechanistic explanations of typical mobile-accessibility issue categories in the following sections.

In sum, the mapping framework establishes a complete “standards-pathways-verification” chain for accessibility, laying the groundwork for lifecycle integration, automated detection, and repair, and signaling a shift of accessibility from an externalized feature toward a core objective of SE.

Table 4: Lifecycle-Based Mapping of Accessibility Responsibilities Across the SDLC

SDLC Phase	Key Accessibility Task	Input → Output (Engineering Deliverables)	Execution Mechanism
Requirements	Model user ability differences & define accessibility constraints	User profiles & task models → Verifiable specifications	Requirements engineering; constraint modeling
Design	Establish semantic & interaction structures compatible with AT	Prototypes & workflows → Traceable interaction & semantic decisions	Interaction design; AT compatibility strategy
Implementation	Implement accessibility APIs & redundant interaction pathways	Codebase → Deployable accessible components & callable APIs	API/ARIA implementation; semantic exposure

SDLC Phase	Key Accessibility Task	Input → Output (Engineering Deliverables)	Execution Mechanism
Testing	Conduct AT-driven accessibility verification	Task models → Pass/fail defect reports & regression outcomes	Automated/AT-driven regression testing
Maintenance	Ensure accessibility continuity across versions & environments	Software updates → Non-regression compatibility validation	Event handling; interface stability assurance

This framework does not assume that standards can be automatically enacted. Instead, accessibility truly becomes a software quality attribute only under explicit engineering prerequisites: at the requirements stage, a verifiable model of ability differences must be produced; at the design stage, traceable semantic and interaction structures must be established; at the implementation stage, accessibility APIs and alternative pathways must be embedded into the software artifact itself; at the testing stage, AT-driven and task-oriented verification mechanisms must be adopted; and at the maintenance stage, interface consistency across devices and versions must be ensured. Without these mechanisms, accessibility remains at the level of ideals and cannot enter the engineering system. Based on this, we systematically map accessibility to the SDLC, transforming it from “implementable standards goals” into “executable process mechanisms” (Table 4). This mapping clarifies each stage’s core tasks, inputs/outputs, and responsibility boundaries, enabling an engineering leap from “can be implemented” to “must be executed.”

(1) Requirements stage: accessibility as a peer engineering input.

Through ability-difference modeling and task analysis, users’ perception and operation constraints are translated into verifiable and traceable requirement specifications, which enter design and testing alongside functional requirements. Outputs at this stage shift from abstract advocacy to engineering-level constraints.

(2) Design stage: structured decisions replace ad-hoc considerations.

Design artifacts must explicitly include semantic structures, focus paths, redundant interactions, and fault-tolerance mechanisms, defining structural boundaries for subsequent code binding and verification. Accessibility thus moves from UI ornamentation to an integral part of system structure.

(3) Implementation stage: accessibility enters the software artifact.

By invoking accessibility APIs, exposing semantic information, and constructing alternative operation pathways, accessibility capabilities are compiled into code and system interfaces rather than attached as external patches. The output is reusable, deployable, and integrable accessible components.

(4) Testing stage: accessibility becomes part of quality assurance.

AT-driven, task-oriented, and regression-based verification identifies defects and generates test results on par with functional defects, giving accessibility quantifiable, regression-ready, and traceable engineering channels.

(5) Maintenance stage: accessibility as a continuously maintained quality attribute.

Through interface-stability validation and environment-compatibility checks, capability degradation is prevented, enabling cross-version evolution and turning accessibility from a one-off task into a long-term maintenance objective.

Through this mapping, accessibility is structurally aligned with the SDLC: architecture provides the hosting locus, while the lifecycle provides the execution path. Accessibility thereby shifts from an optional UI-layer feature to a full-process engineering responsibility, strictly matching SE’s core logic-requirements-driven development, structural decision-making, implementation binding, quality verification, and version evolution.

Taken together, Fig. 4 and Table 4 demonstrate that accessibility is not an external add-on, but an internal quality objective. Fig. 4 provides the capability structure, while Table 4 provides the lifecycle mechanisms. Based on these, our study for the first time operationalizes accessibility as a modelable,

implementable, verifiable, and maintainable SE attribute, filling the long-standing structural gap between standards and practice.

4.2 Current Status of Mobile Application Accessibility Issues

To systematically summarize the state of research on mobile application accessibility issues, we analyzed 39 relevant studies and, drawing on WCAG's four core principles, constructed an issue taxonomy tailored to mobile contexts. This taxonomy integrates the mapping model above and absorbs recurrent barrier scenarios from user studies and technical practice, aiming to build an analyzable framework that is mappable between theoretical abstractions and real user experience. Large-scale empirical evidence indicates that nearly all mobile apps exhibit accessibility defects to varying degrees, and these defects commonly cause substantial difficulties for users with visual, hearing, motor, and cognitive impairments [69]. Therefore, a scientific and systematic issue taxonomy is essential not only for locating weak links in accessibility practice, but also for advancing mobile-accessibility research. Based on the above evidence and interaction model, we summarize typical accessibility issues into four categories—corresponding to information perceivability, interface operability, interaction understandability, and system robustness—and analyze their impacts on different ability groups.

(1) Perceivable problems (Perceivable).

This category primarily affects users' ability to obtain and recognize UI content, including textual information, images, icons, button states, and so on. The most impacted groups include users with visual impairments, color-vision deficiencies, cognitive impairments, and motor impairments. For example, Chen et al. [23] focused on assessing accessibility levels of Android apps. They analyzed 1,000 popular apps from Google Play across 33 mobile app categories. Using a combination of automated tools and manual inspection, they identified and classified 11 major accessibility issues, such as insufficient text contrast, overly small clickable areas, and missing screen-readable text. Their results show that 46.25% of mobile apps have at least one severe accessibility issue, and about 70% lack content descriptions that are critical for users with visual impairments. This large-scale analysis reveals the severe state of Android accessibility and underscores the need to pay greater attention to accessibility design and testing during mobile app development. Another study on 12,000 Android apps collected from Google Play found that most Android apps suffer from serious accessibility issues [54]: 46.25% contain at least one defect related to “screen-readable text,” while 60.91% present problems with “touch target size.” Even critical government service apps are not exempt. Mateus et al. [55], investigating the accessibility of Brazilian government mobile apps (whose usage increased sharply at the time), applied the MATE tool [56] to automatically test 15 mobile apps in three groups: health, public banking, and social/information services. They categorized detected issues according to WCAG and BBC mobile accessibility standards. Their findings show that many government apps still have severe accessibility problems, including untagged CAPTCHA images, unlabeled form fields, insufficient color contrast, and inaccessible buttons and UI activators. In particular, low image contrast is the most frequent issue, accounting for over 30%. Mateus et al. further observed that many dynamic content changes cannot be perceived by screen readers, causing users with visual impairments to miss important information [70]. From nearly 5,000 disability-related reviews extracted from Google Play, Oliveira et al. found that overly small fonts, indistinguishable icons, and weak color contrast are the most salient complaint types [71].

These issues significantly reduce users' ability to understand interface structure and content hierarchy, preventing them from completing basic information-perception tasks. They are the most common and foundational issue type in current accessibility design. Table 5 details the main mobile-accessibility issues under information perceivability.

Table 5: Mechanism-Based Classification of Mobile Perceivability Problems and Their Mapping to Engineering Capability Domains

Category	No.	Specific Problem	Affected Users	Underlying Mechanism	Mapped Engineering Capability Domain
Missing Semantic Exposure and Metadata	①	Missing content descriptions/labels	Visual impairment	Elements lack semantic names, preventing AT from conveying meaning	Semantic Structure & Content Exposure
	②	Unidentifiable icons lacking semantic description	Visual & cognitive impairment	Abstract icons without textual or semantic cues	Semantic Structure & Content Exposure
	③	Unnamed custom components	Visual impairment	Custom UI not registered to accessibility API	API Compatibility & Semantic Binding
Insufficient Visual Contrast and Legibility	④	Low text contrast	Low vision	Foreground-background luminance insufficient	Information Perception (Contrast Handling)
	⑤	Low image/icon contrast	Low vision & color vision deficiency	Visual boundaries unclear due to poor contrast	Information Perception
	⑥	Elements visually too small	Visual & motor impairment	Visual targets difficult to perceive	Interface Operability (Visual Perception)
Inadequate Scalable and Adaptive Presentation	⑦	Text too small or non-resizable	Low vision	Typography not scalable or ignoring system font scaling	Visual Adaptation & Interface Operability
	⑧	View cannot zoom or breaks after zoom	Visual impairment	Layout not responsive to scaling; leads to clipping/overlap	Compatibility & Visual Robustness
	⑨	View content missing after text enlargement	Visual impairment	Layout fails to reflow under larger font sizes	Layout Adaptation & Robust Rendering
Non-Perceivable Dynamic or Modal Changes	⑩	Dynamic changes not announced	Visual impairment	State changes not broadcast to AT	Event Management & State Notification
Lack of Multisensory or Alternative Perception Channels	⑪	AR content lacks descriptive alternatives	Visual impairment	No audio/haptic alternatives for visual-only content	Multimodal Support
Spatial Density and Perceptual Confusion	⑫	Spacing too small or too large	Visual & motor impairment	Irregular spacing causes visual confusion or misinterpretation	Interaction Logic & Visual Layout Organization

Note: Supporting evidence for each problem is provided in Appendix A.

(2) Operable problems (Operable)

This category concerns barriers to users' ability to interact with and operate the interface, including input control, navigation flows, and multi-finger operations. Typical issues are summarized in Table 6. The primary affected groups are users with motor impairments and users with temporary hand constraints (e.g., fractures or one-handed operation). Lopes et al. reported through manual testing that, in commonly used banking apps, interactive widgets are densely arranged and focus transitions are disordered, making it difficult for TalkBack users to complete basic task flows [58]. Mateus et al. further found that among issues missed by automated tools, about 75% are related to user operation workflows and can only be identified via real user testing [59].

Such issues often do not prevent information reading, but they severely disrupt actual interaction processes, and are among the hardest accessibility problem types to capture automatically. Table 6 details the major operability problems observed in mobile apps.

Table 6: Mechanism-Based Classification of Mobile Operability Problems and Their Mapping to Engineering Capability Domains

Category	No.	Specific Problem	Affected Users	Underlying Mechanism	Mapped Engineering Capability Domain
Focus and Reachability Failures	①	Inaccessible hyperlinks	Visual impairment	Hyperlinks not exposed as operable elements; unreachable via AT focus	Focus Navigation & Interface Operability
	②	Insufficient focusable elements	Visual impairment	Key interactive components lack focus paths	Focus Navigation
	③	Elements cannot be focused (e.g., buttons, input fields)	Visual impairment	Components not registered to AT focus model	API Compatibility & Focus Binding
Activation and Trigger Mechanism Failures	④	Focused elements cannot be activated	Visual impairment	Gesture-to-action mapping not supported (e.g., double-tap not triggering action)	Event Management & Action Activation
	⑤	Unclear interactive state (enabled/disabled)	Visual impairment	State not semantically exposed; focusable but non-operable	Semantic State Exposure & Interaction Logic
Target Size and Precision Constraints	⑥	Touch targets too small	Motor impairment	Operable regions below touch threshold; high precision required	Interface Operability (Target Sizing)
Input Modality and Gesture Limitations	⑦	Complex gestures without alternatives	Visual, motor, cognitive impairment	Multi-finger/complex gestures required; no simpler substitute operations	Multimodal Support & Alternative Input Pathways
Interaction Noise and Irrelevant Elements	⑧	Non-operable elements announced as operable	All user groups	Non-actionable content exposed as interactive targets, increasing navigation burden	Interaction Logic & Semantic Filtering

Note: Supporting references corresponding to each listed issue are detailed in Appendix B.

(3) Understandable problems (Understandable)

These problems relate to the cognitive difficulty of interface logic, terminology, interaction flows, and feedback mechanisms. Typical issues are listed in Table 7. The most affected groups include users with cognitive impairments, reading difficulties, neurodiverse users (e.g., autism spectrum), and language learners; among them, users with cognitive impairments and autism spectrum conditions are particularly sensitive to this class of issues. Akter et al. found that vague error prompts and unclear operational logic often lead screen-reader users to perform incorrect operations, or even abandon the mobile app altogether [81]. Serra et al. observed that several government apps lack explicit feedback after form validation failures, significantly reducing users’ task completion rates and trust [57].

Although these issues are difficult to quantify and detect, they exert substantial impact on users’ operational confidence and task success, and constitute a key bottleneck for accessibility to move from mere “compliance” toward genuine “usability.” Table 7 details the major understandable problems in mobile apps.

Table 7 Mechanism-Based Classification of Mobile Understandability Problems and Their Mapping to Engineering Capability Domains

Category	No.	Specific Problem	Affected Users	Underlying Mechanism	Mapped Engineering Capability Domain
Semantic Ambiguity and Redundancy	①	Duplicate labels for multiple elements	Visual impairment	Identical labels prevent AT from distinguishing functions	Semantic Structure & Content Disambiguation
	②	Redundant or low-value information	Visual & cognitive impairment	Excessive verbal output increases cognitive load	Information Perception & Semantic Filtering
Disordered Navigation and Unpredictable Interaction Flow	③	Complex or inconsistent navigation order	Visual & cognitive impairment	Focus sequence misaligned with visual/mental model; unpredictable transitions	Interaction Logic & Focus Navigation
Information Overload and High Cognitive Density	④	Excessive information density	Visual & cognitive impairment	Dense layouts hinder extraction of key information	Interface Organization & Cognitive Load Management
Unannounced State Changes	⑤	Unclear element states (e.g., switch on/off)	Visual & cognitive impairment	State changes not exposed through AT or semantic cues	State Notification & Event Management
Excessive Interaction Steps	⑥	Overly complex task flows	Cognitive impairment	Multiple steps or repeated inputs increase cognitive burden	Interaction Logic & Workflow Simplification
Ambiguous or Missing Error Feedback	⑦	Unclear or unread error messages	All user groups	Error states not semantically exposed or not announced via AT	State Notification & Error Communication

Note: Evidence sources for the understandable problems are included in Appendix C.

(4) Robustness problems (Robustness)

This category concerns the sustained availability and compatibility of mobile accessibility across platforms, versions, devices, and AT, serving as the underlying support that determines whether accessibility mechanisms can run stably. Typical issues are shown in Table 8. The primary affected groups are users who rely on AT (e.g., screen readers, voice control, switch devices), especially users with visual impairments and those with high dependence on assistive services[97].

Software updates should ideally create opportunities to improve UX and repair accessibility defects. However, a growing body of evidence indicates that updates frequently trigger “accessibility regression” : previously supported accessibility features are removed, interaction logic becomes disordered, or AT compatibility breaks, which severely undermines the continuity of use and accessibility guarantees for disabled users. This phenomenon reflects insufficient robustness in current systems under version evolution, and can further induce cascading degradation in key dimensions such as operability and understandability.

From a technical-mechanism perspective, Huq et al. [17] emphasize that certain updates, while optimizing visual design, can damage semantic annotations and focus structures, preventing screen-reader users from completing previously workable workflows. Moreover, accessibility behaviors may vary across visual themes, exposing weaknesses in robustness of development and support tooling. Existing tools often exhibit interface rendering confusion or assistive-feature failures under dark mode. Ma et al. [82] thus call for explicit consideration of dark mode when developing support tools, implying a need for stronger robustness to accommodate different visual themes.

From a user perspective, Oliveira et al. [95] analyzed 694 Google Play reviews that reported post-update accessibility issues, and manually coded them under WCAG 2.2. They found that about 52% of comments

involved perceivable-principle issues, especially insufficient color contrast, overly small fonts, and missing descriptions. Some users reported that accessibility features were removed in newer versions (e.g., magnification or voice-feedback buttons), increasing usage difficulty and indicating frequent accessibility regression during software evolution. Santos et al. [84] further corroborate this trend: among 4,999 visual-impairment-related accessibility reviews from Google Play, 694 were update-related, with 93% being negative and explicitly indicating degraded accessibility after updates. Reported issues include incompatibility with screen readers, removal of accessibility features (e.g., color schemes or font customization), overly small fonts and widgets, and incompatibility with device-level accessibility configurations. These barriers substantially impair users' perception, understanding, and operation.

In addition, fragmentation and implementation differences in platform standards are another major contributor to robustness failures. Across platforms (e.g., Android vs. iOS), OS versions, or regional variants, accessibility API coverage, AT response logic, and semantic-labeling mechanisms often diverge. Mobile apps may perform well on one platform yet become hard to use on another [98][99]. Compounding this, mainstream accessibility evaluation tools lack unified standards and transparent scoring, frequently yielding situations where “one platform passes while another fails.” Such cross-platform inconsistency and non-uniform standards increase adaptation costs for developers and weaken the sustained usability and evaluability of accessibility in multi-platform ecosystems, forming a systemic robustness challenge.

Aljedaani et al. [100], using the Chromium project as a case study, conducted an empirical analysis of mobile-platform accessibility issue reports to examine developers' and organizations' responses. They found that although accessibility reports have increased in recent years, their repair cycles are generally long, largely because these issues are often assigned low priority. Moreover, Android exhibits significantly more accessibility issues than iOS, with a rising trend over time, whereas iOS-related issues decline annually. Across all issue reports, only 4% were accessibility-related—3% for Android and 1% for iOS—while the remaining 96% concerned other issue types. The study suggests that Android's higher counts may stem from its more diverse device ecosystem, larger user base, and higher Chromium release frequency. Rapid iteration can introduce new accessibility issues within short development and evaluation cycles. As the population of AT users grows and related reports continue to rise, the authors argue that developers must invest more resources to systematically identify and resolve these issues, improving overall accessibility.

Robustness problems are difficult to uncover through static detection alone; they are often buried in processes such as version changes, framework replacement, or API shifts, and thus require version-comparison testing and AT regression testing to capture. The core challenge is maintaining accessibility consistency and stability under system evolution—an SE quality-assurance gap that urgently needs strengthening.

Table 8 Mechanism-Based Classification of Mobile Robustness Problems and Their Mapping to Engineering Capability Domains

Category	No.	Specific Problem	Affected Users	Underlying Mechanism	Mapped Engineering Capability Domain
Visual Theme and Contrast Instability	①	Improper dark mode design	Visual impairment	Color contrast and visual rendering not preserved under theme changes	Compatibility & Visual Robustness
Failure to Honor System-Level Accessibility Settings	②	Incompatibility with device accessibility configurations	All user groups	Application does not respond to OS-level preferences (e.g., high contrast, large fonts)	API Compatibility & System Preference Integration

Category	No.	Specific Problem	Affected Users	Underlying Mechanism	Mapped Engineering Capability Domain
Cross-Device and Cross-Platform Inconsistency	③	Inconsistent accessibility across devices/platforms	All user groups	UI behavior and accessibility exposure vary across device models/frameworks	Cross-Platform Compatibility & Interface Consistency
Regression and Feature Removal	④	Removal of accessibility features after updates	All user groups	Version updates break or remove previously supported accessibility capabilities	Version Resilience & Non-Regression Assurance
Embedded Third-Party Content Fragility	⑤	Inaccessible nested advertisements	All user groups	Third-party ad components lack semantic labels, operable controls, and predictable exits	Event Management & Interaction Safeguards
AT Integration and Cooperation Failures	⑥	Poor compatibility with assistive technologies	All user groups	Application fails to expose states/actions reliably to AT (e.g., TalkBack/VoiceOver)	AT Integration & API Compatibility

Note: Supporting citations for robustness-related issues are provided in Appendix D.

Building on WCAG’s four principles and the standards-to-practice mapping model, and integrating user feedback with empirical testing evidence, this section systematically classifies mobile accessibility issues. The taxonomy reveals usage challenges faced by four major ability-limited groups—visual, hearing, motor, and cognitive impairments [25][68][102] (older adults can be subsumed into these categories due to ability degradation [103])—and provides a structured foundation for subsequent automated detection, differentiated repair strategies, and SDLC-oriented engineering integration.

5 ANSWER TO RQ2: WHAT IS THE EVOLUTIONARY TRAJECTORY OF MOBILE APPLICATION ACCESSIBILITY DETECTION TECHNIQUES, AND WHAT COVERAGE, AUTOMATION LEVEL, STRENGTHS/LIMITATIONS, AND APPLICABLE SCENARIOS DOES EACH LINE OF WORK EXHIBIT?

Building on the analysis of 20 representative detection studies, this section systematically reviews the evolution of mobile application accessibility detection. Prior taxonomies often start from “manual-automated-user testing” or “static-dynamic,” and/or “development-time-runtime.” In practice, however, these boundaries frequently overlap, and thus fail to capture tool-level differences in detection capability, semantic understanding, and integration depth. By jointly considering detection mechanisms, analysis strategies, and levels of intelligence, we identify five categories: static analysis, manual dynamic testing, framework-integrated dynamic testing, automated GUI exploration and behavior analysis, and AT simulation with AI-enhanced testing. This taxonomy more directly reveals the evolutionary path and suitable scenarios of mobile accessibility detection. Compared with traditional stage- or strategy-centric classifications, it better exposes differences across methods in coverage, automation, and engineering integration, providing an actionable reference framework for future research and systematic toolchain construction.

5.1 Static Analysis

Static analysis is among the earliest and most fundamental approaches to mobile accessibility detection. It identifies violations of accessibility design guidelines by analyzing source code, layout files, and resource configurations, without running the app, and is therefore suitable for early development stages [45]. Such methods help developers surface potential issues during coding, reduce late-stage rework cost, and improve overall code quality and maintainability.

In industry, Android Lint [104] is one of the earliest static tools with accessibility checks. Introduced by Google in 2011 and deeply integrated into Android Studio, it performs comprehensive analysis over project structures, including Java/Kotlin code, XML layouts, and resource directories. For accessibility, Lint mainly checks:

- Whether visual components provide appropriate content descriptions (e.g., missing `contentDescription`);
- Whether input fields are correctly labeled and grouped so that AT can identify their purpose;
- Whether custom view components override AT-required accessibility interfaces and methods[105];
- Other common code-quality issues, such as missing localized strings or layout performance defects that affect accessibility.

Fig. 5 shows the basic architecture and workflow of static detection.

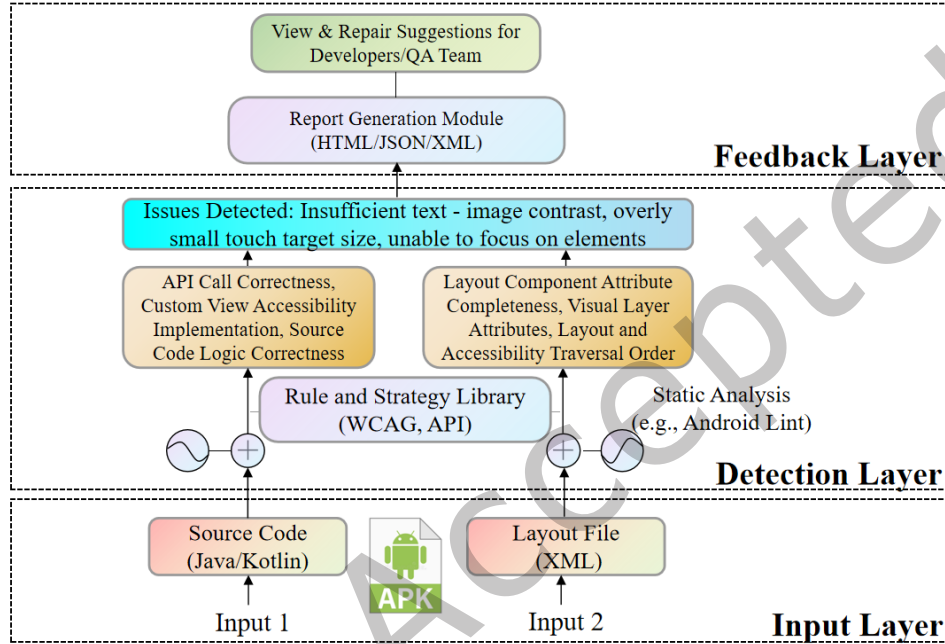


Fig. 5 Architecture of static accessibility detection

Although static detection provides a fast and efficient early-stage localization mechanism, it has inherent limitations [23][49][50][52][56]. First, static analysis only processes properties derivable from code artifacts, and thus cannot detect runtime-only accessibility problems—for example, whether touch target size meets minimum accessibility thresholds; whether color contrast remains compliant across screen densities and visual themes; or whether dynamically loaded components are correctly registered to AT during execution. Second, over-reliance on static detection can create a false sense of safety, because key issues—such as whether widgets are focusable in real runtime contexts, or whether semantic information is lost due to contextual changes—only surface through simulated or real user interactions. Hence, while static detection is indispensable as a starting point, it cannot cover all problems and must be combined with dynamic methods to form a more complete detection system.

Beyond Android Lint, academia has proposed multiple research prototypes for static accessibility analysis on mobile, expanding rule coverage to dimensions such as UI hierarchy depth, semantic completeness of labels, and validity of resource references. These advances continue to push static approaches toward stronger semantic understanding and rule generalization, providing a solid foundation for later integrated automated frameworks.

5.2 Manual Dynamic Testing

Manual dynamic testing is one of the earliest methods used for mobile accessibility evaluation. It typically requires developers or testers to operate a running mobile app with AT and manually verify the accessibility properties of UI components [46]. This approach is highly accurate, especially for semantic errors, context-dependent problems, and defects in dynamically generated content, but it is inefficient and knowledge-intensive, and thus mainly serves as supplementary validation for critical or high-priority modules.

A typical manual workflow includes: navigating screen by screen, activating a checker on the current screen, scanning for accessibility issues, and obtaining instant feedback on accessibility properties of UI elements. Most tools provide screen-level checks and hints; their basic architecture is illustrated in Fig. 6.

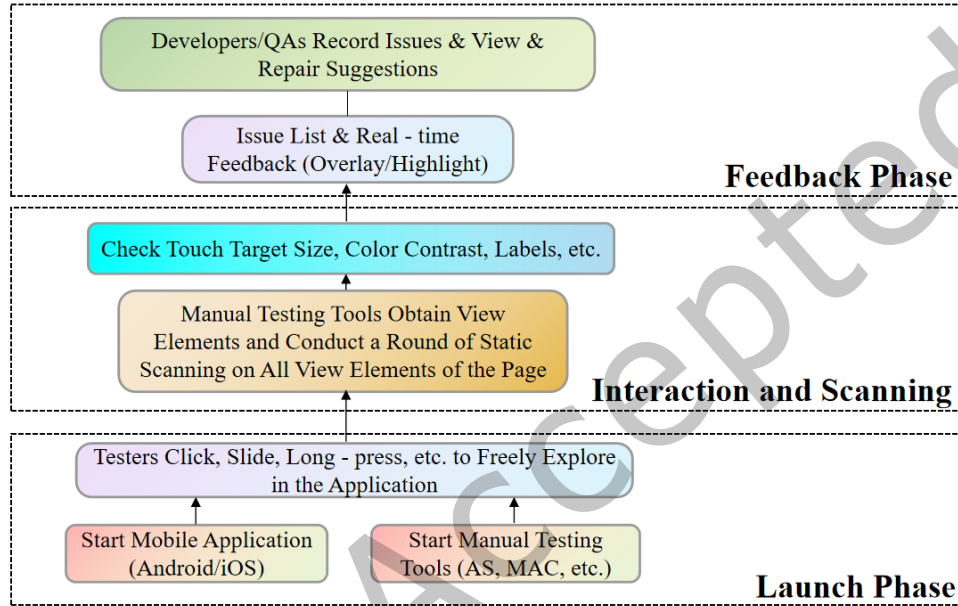


Fig. 6 Architecture of manual dynamic testing

With the maturation of mobile development environments, representative manual tools have emerged in both industry and academia, including:

- **Accessibility Scanner (AS) (Android)** [36]: a Google-developed mobile app that provides real-time accessibility improvement suggestions. Users must activate it manually and scan each screen. Checks include missing alternative text for non-text elements (e.g., `contentDescription`), insufficient contrast, too-small touch targets, duplicate labels, and overlapping operable elements. These checks are implemented via Google's open-source Accessibility Test Framework (ATF) [106], which supports multiple accessibility rules on Android.
- **Accessibility Inspector (iOS)** [35]: Apple's native tool that exposes accessibility properties of all UI elements in an app, including labels, values, and hints. Developers can inspect elements one by one to identify missing labels or inconsistencies, or enable built-in validation to automatically flag issues such as non-text elements lacking alternative descriptions.
- **IBM AbilityLab Mobile Accessibility Checker (MAC)** [107]: supports both iOS and Android and can be integrated into development environments to detect accessibility issues early. It checks for missing descriptions, insufficient contrast, improper element sizing, and disordered focus logic. During user interaction, it captures issues and generates feedback reports, strengthening accessibility awareness during development.
- **UI Automator Viewer (Android)** [108]: part of the Android SDK that captures UI hierarchies from real devices or emulators and provides detailed properties such as coordinates, size, class name, resource ID, and content descriptions. Patil et al. [109] enhanced it into an improved UI Automator Viewer that supports multi-device parallel checking and integrates touch/key event recording and replay, enabling "semi-automated" dynamic testing. The added contrast check further broadens its applicability.

Despite their direct usefulness, manual tools have notable limitations. Studies show that manual testing is inherently “labor-intensive and time-consuming,” requiring screen-by-screen operation and repeated tool activation [23][45][110]. For large, complex, or frequently updated apps, re-evaluating all screens is impractical, lacking scalability and long-term maintainability.

Moreover, manual tools usually cannot provide global reports; most only generate feedback for the current activity screen, preventing developers from understanding app-wide issue distributions and patterns. This “screen-level feedback” lacks contextual memory [51]. Huq et al. [17] also note that such tools may generate redundant error logs, increasing cognitive load for issue triage.

Therefore, although manual testing still plays an important role in specific scenarios—particularly for in-depth diagnosis of critical flows or as a complementary validation step to other methods—its limitations have also motivated researchers to explore new directions. On the one hand, some studies extend manual dynamic testing with runtime instrumentation and crowdsourcing ideas, shifting accessibility checks from test-device environments to real end-user devices. For example, Barboza et al. [111] proposed the VisioAux framework, which integrates an accessibility-testing SDK into mobile apps so that, during end-users’ everyday interactions, ATF rules are automatically invoked on their devices and the resulting findings from multiple devices are aggregated for analysis, providing a new engineering perspective for building continuous accessibility monitoring mechanisms.

On the other hand, in order to perform accessibility detection in a more controllable and systematic way during development and testing, and to further improve automation and reusability, the research community has increasingly explored more efficient and scalable techniques such as framework-integrated dynamic testing and automated GUI exploration. These techniques lay the groundwork for constructing more systematic accessibility testing workflows.

5.3 Framework-Integrated Dynamic Testing

Framework-integrated dynamic testing embeds accessibility checks into existing GUI automation test pipelines, enabling rules to be triggered and executed automatically. Compared to manual testing, these frameworks can more systematically and efficiently localize runtime accessibility problems, and are particularly suitable for CI environments with rapid iteration demands [48].

Typically, developers write functional/system test scripts; when the mobile app executes predefined interaction tasks, the framework automatically evaluates accessibility attributes of involved components. Once violations are detected (e.g., non-compliance with WCAG), testing is interrupted and exceptions or reports are generated, prompting early repairs. The basic principle is shown in Fig. 7.

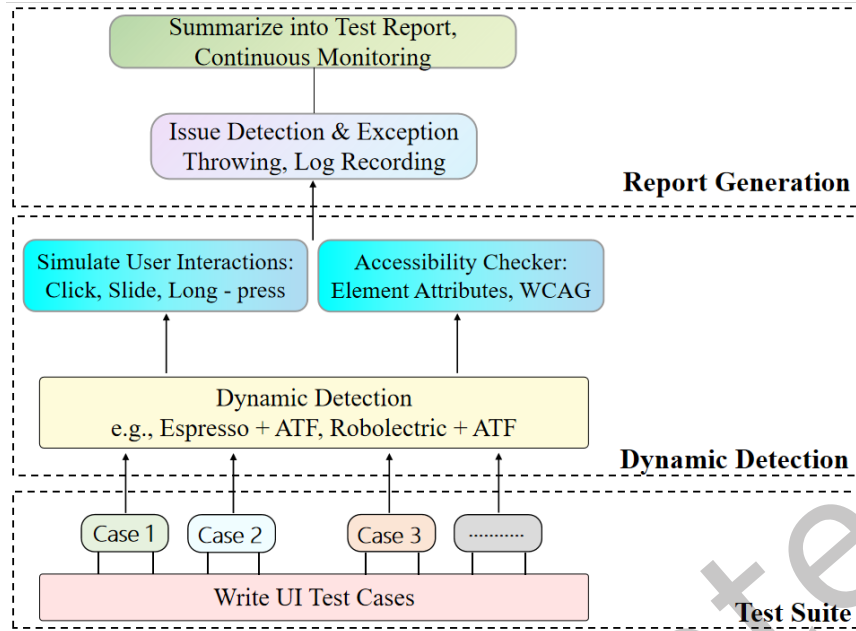


Fig. 7 Principle of framework-integrated dynamic detection

In the Android ecosystem, two representative frameworks are:

- Espresso [112]: Google’s Android GUI automation framework that supports complex interaction simulation. Developers can embed accessibility rules in test classes; non-compliant elements are reported automatically. Espresso integrates ATF under the hood, using the same rules as AS, and checks common issues such as missing labels, improper target sizing, and insufficient contrast.
- Robolectric [113]: an Android unit-testing framework running on the JVM. It is more efficient than Espresso but operates in a simulated activity environment rather than real devices, limiting its ability to detect issues that only appear in real contexts. Its accessibility checks also rely on ATF and are similar to Espresso’s [53].

On iOS, integrated solutions include:

- EarlGrey [114]: Google’s open-source “iOS Espresso,” running inside the mobile app process with strong synchronization and extensibility. It can locate elements inside scroll containers, improving adaptability to dynamic UI, and integrates seamlessly with Apple’s XCTest. After tests, it performs WCAG-based rule analysis over the UI hierarchy and outputs detailed reports [115].
- KIF (Keep It Functional) [116]: another widely used iOS framework that simulates touch, swipe, and text input. It integrates into Xcode and calls native interfaces for accessibility checks, reducing middleware dependencies and improving performance. Its philosophy resembles Robolectric, emphasizing realistic automated inputs and lightweight execution.

These frameworks significantly improve the systematization and scalability of dynamic detection, realizing “testing-as-detection.” Yet they face substantial challenges [23][50]. First, integration costs are high: developers must script tests for UI components and insert relevant APIs, increasing workload. Second, effectiveness depends heavily on the quality and coverage of test suites [117]. If tests cover only typical paths, many issues are missed. Empirical studies report that only about 14% of mobile apps include test cases, and only roughly 9% exceed 40% coverage [118][119][120]. Under such conditions, requiring dedicated accessibility cases for all components is hard to scale, and some frameworks (e.g., Robolectric) may find no accessibility issues in practice [121][122].

Thus, while framework-integrated testing is conceptually and practically advantageous—especially when paired with CI—it still suffers from heavy test burdens and low automated coverage, and benefits from being combined with higher-automation methods such as automated GUI exploration.

5.4 Automated GUI Exploration and Behavior Analysis

Automated GUI exploration represents a major paradigm shift in mobile accessibility detection. Its goal is to systematically activate mobile app screens via automated traversal, and then capture runtime accessibility problems by analyzing UI states and response behaviors [23]. Compared with static rule checks or test-case-driven dynamic testing, this approach removes the prerequisite of scripted tests and avoids the time and scalability bottlenecks of manual testing. Fig. 8 illustrates its basic framework.

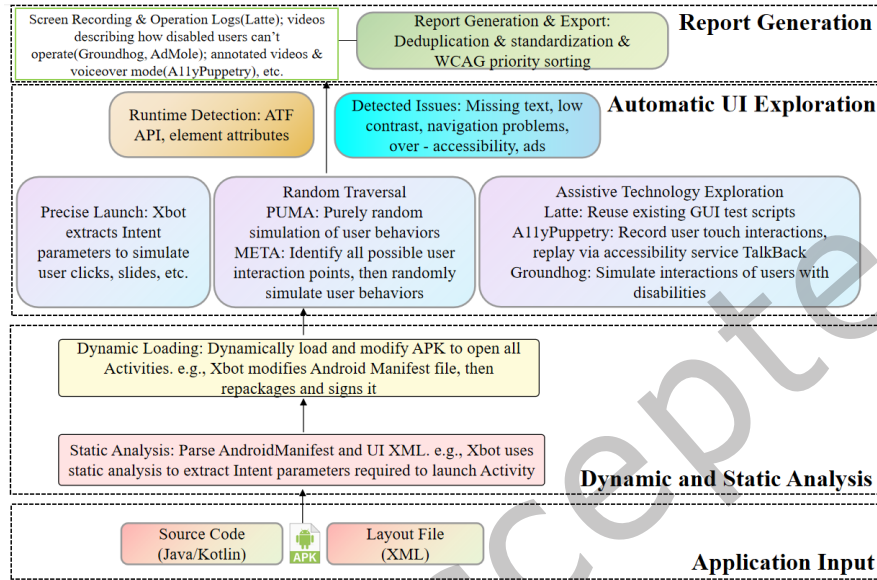


Fig. 8 Framework of automated GUI detection

To enhance flexibility and reuse, PUMA by Hao et al. [123] decouples exploration logic from analysis logic. Using event generators such as Google Monkey [124][125], PUMA simulates clicks, swipes, and text input to explore execution paths, after which the analysis module checks properties of specific UI states (e.g., missing descriptions, contrast violations, non-compliant touch targets). This modularity improves scalability and cross-app reuse.

To address missing test suites, Eler et al. [56] proposed MATE, emphasizing zero-script automated test generation. Running on emulators or real devices, MATE uses UI Automator to identify interactive elements and randomly triggers interactions while performing multiple accessibility checks, detecting issues such as duplicate elements, unlabeled components, and abnormal contrast. It also handles dynamic components and produces detailed reports.

However, MATE's exploration is largely heuristic and random, making it hard to reach flows requiring login, complex inputs, or special permissions [23][69]. To overcome this, Chen et al. proposed Xbot [23], which extracts precise Intent parameters needed to launch activities via instrumentation and static analysis, then performs dynamic accessibility checks on activity pages, achieving up to 80% activity coverage. Xbot's multi-stage pipeline includes APK instrumentation and repackaging, Intent extraction, and ATF-based detection, substantially improving precision.

Although MATE and Xbot achieve strong coverage for common issues, they often fail to simulate realistic AT user trajectories, and thus miss experiential problems such as missing focus or assistive-service trigger failures [50][110]. Researchers therefore began to simulate AT behaviors. To better approximate real disabled-user contexts, Salehnamadi et al. [50] proposed Latte, which reuses existing GUI functional test scripts and replays them with TalkBack, SwitchAccess, and other assistive services. Latte follows three stages: extracting natural-language test specifications, executing interactions via AT simulation, and producing reports—reducing developer burden while improving accuracy.

Latte's coverage, however, depends on the existence of functional tests, which only about 14% of mobile apps have [118][126]. To mitigate this, Alotaibi et al. [127] proposed ATARI, constructing a TalkBack Swipe Navigation Graph (TSNG) and a Touch Navigation Graph (TNG), then detecting nodes reachable by touch but inaccessible via TalkBack, thereby identifying TalkBack-specific interaction failures (TIAFs).

To balance AT realism with higher automation, Salehnamadi et al. [110] introduced A11yPuppetry, using a "record-translate-replay" pipeline to convert real user interactions into TalkBack-recognizable operations, supporting multiple navigation modes (linear, jump, search, touch) and producing visualized reports. Yet it remains sensitive to UI version changes and incurs high maintenance cost [117][120].

Groundhog [128] removes reliance on tests by crawling apps in a virtual environment, simulating touch and TalkBack interactions to assess operability and accessibility, and producing video summaries that vividly demonstrate real barriers. While stronger than Latte and ATARI, Groundhog has limited support for advanced gestures and dynamic content, and struggles to reproduce some complex UI states [77][129].

In the ad-screen accessibility domain, He et al. [78] extended Groundhog into AdMole, drawing on ad-request mapping techniques [130][131]. AdMole targets five issue types in ads: linear navigation failures, touch failures, unlabeled elements, invalid clicks, and excessive interaction, filling an empirical gap in ad accessibility.

For detecting accessibility issues under text-scaling services, AccessiText [132] runs two rounds of GUI tests (default vs. enlarged fonts) and combines screenshots with metadata to identify missing/overlapping/clipped/truncated layouts after scaling, achieving over 88% precision and recall. Su et al.'s dVermin [133] models pre-/post-scaling UI differences using connected-component analysis and SSIM to localize issues more precisely, though at higher time cost.

Notably, most work focuses on insufficient accessibility, whereas Mehralian et al. [89] proposed OverSight to study over-accessibility risks—where AT users can access hidden functions or sensitive information that are unavailable to sighted users. OverSight combines static analysis and dynamic validation to detect Android OAsms¹¹.

OverSight validates such smells via TalkBack and SuperAT, revealing structural tension between accessibility and security: systems must broaden support for diverse abilities while preventing misuse or abuse of accessibility mechanisms. This suggests that future accessibility design must dynamically balance "improving accessibility" and "controlling access boundaries," ensuring system robustness and security while pursuing fairness.

Despite continuous gains in coverage and precision, two bottlenecks remain common. First, most detection algorithms depend on predefined rules and cannot handle context-dependent issues such as reading order or semantic tailoring. Second, they lack sufficient human-like understanding to judge true usability under AT. Consequently, deep-learning and LLM-based detection is emerging, aiming to inject semantic modeling and interaction learning into automated accessibility testing and bridge the gap between automation and human cognition.

5.5 AT Simulation with AI-Enhanced Testing

AT simulation with AI-enhanced testing is becoming a key direction for improving the depth and breadth of mobile accessibility detection. These methods combine AI techniques—such as deep learning, reinforcement learning, and NLP—to simulate realistic AT user paths or to detect and explain accessibility

¹¹ OAsms (Overly Accessibility and Operability smells) refer to issues like "excessive accessibility design" and "operability design smells" in Android applications. The former means the lack of access control in accessibility design, allowing AT users to access interfaces, functions, or data that should not be exposed (e.g., manipulating background services or unauthorizedly modifying settings via accessibility services). The latter refers to potential defects in operability design, especially interaction risks or unauthorized access hidden in accessibility paths (e.g., interfaces disabled in UI but still operable by AT). The term "smell" originates from "design smell" in software engineering, referring to design indications that may hide structural defects or potential risks without necessarily causing errors.

issues with stronger context awareness via semantic modeling and pattern recognition. Some studies further integrate model training into issue categorization and prioritization, showing promising generalization and adaptability [50][52].

Deep learning, as a transformative technology, enables a paradigm shift from rigid rule-based testing to more intelligent and adaptive solutions. Deep models—especially CNNs and LLMs—excel at processing unstructured data such as images, text, and audio. CNNs can analyze UI screenshots to recognize visual elements and design patterns, and precisely localize subtle display anomalies such as text overlap, occlusion, clipping, or missing images—tasks that are extremely challenging for rule-based methods. The overall framework of deep-learning-enhanced detection is shown in Fig. 9.

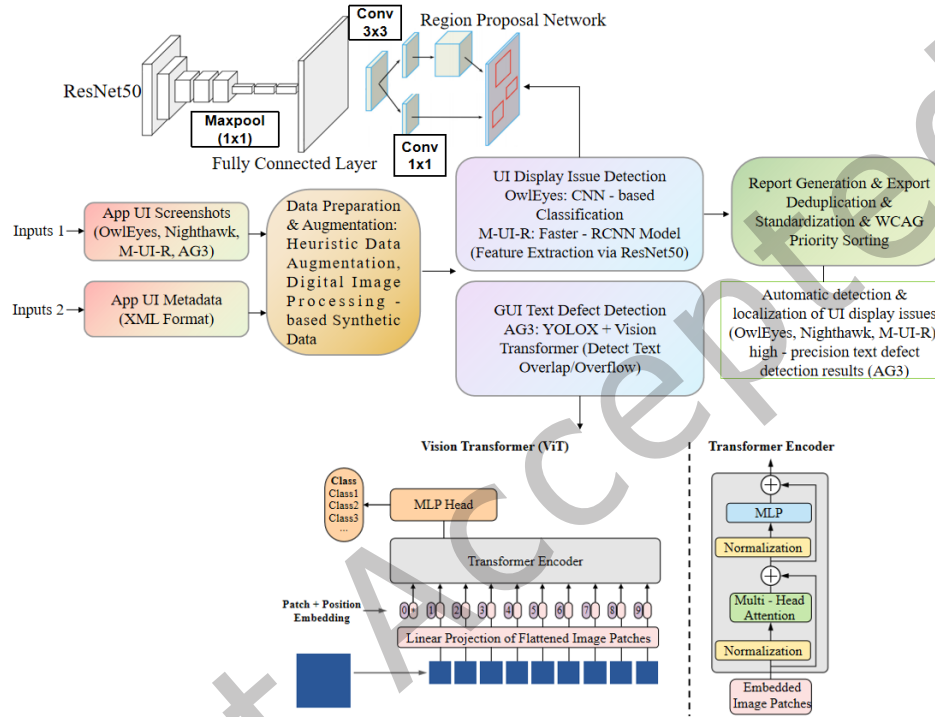


Fig. 9 Framework of deep-learning-enhanced detection

LLMs provide human-like language understanding and generation, enabling them to interpret natural-language testing instructions, analyze semantic errors in screen-reader outputs, and produce context-aware explanations and repair suggestions. The application framework of LLMs in accessibility detection is shown in Fig. 10.

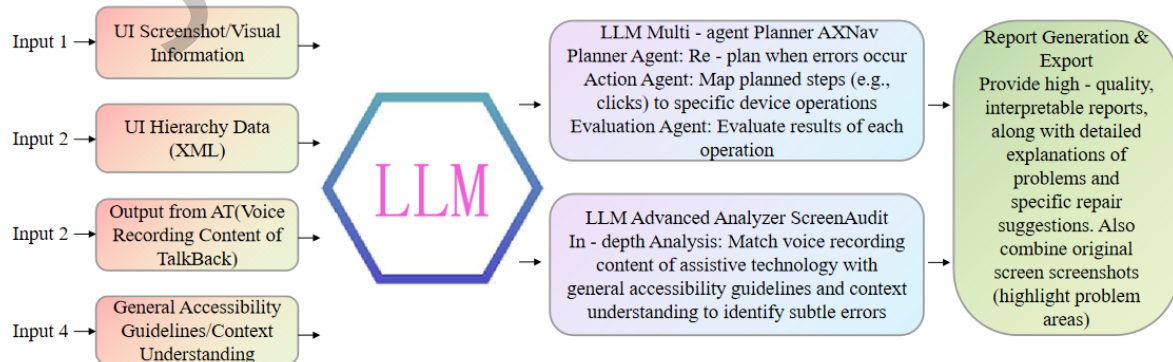


Fig. 10 Application framework of LLMs in accessibility detection

5.5.1 Breakthroughs in Deep-Learning-Based UI Display Defect Detection

To address limitations of conventional GUI testing in capturing UI display defects—and the cost and incompleteness of manual labeling—Liu et al. [134] proposed OwlEyes. OwlEyes uses CNNs to classify defective screenshots and applies Grad-CAM heatmaps to localize problematic regions, detecting occlusion, text overlap, and missing images. To mitigate data scarcity, OwlEyes synthesizes defective images by parsing JSON metadata of clean screenshots and perturbing positions/sizes of components such as TextView and ImageView. However, localization regions are often coarse and thus provide limited precision for repair.

To further improve precision, Liu et al. [135] proposed Nighthawk, which builds an object-detection model using Faster-RCNN and ResNet50 to precisely detect and localize UI display defects. It automatically generates labeled synthetic data heuristically, alleviating training-data shortages. Nighthawk outperforms OwlEyes in accuracy and cross-platform adaptability, standing as a representative system for UI display defect detection.

Ali et al. [136] also proposed M-UI-R, leveraging Faster-RCNN to detect issues such as text overlap and component occlusion, achieving an average precision of 87.7%—about 16% higher than OwlEyes. While effective for mainstream UI display defects, it still struggles with tiny defect regions and does not yet cover issues like button sizing or layout rationality.

5.5.2 AI-Driven Detection for Specific Accessibility Issues

Accessibility detection is also becoming more specialized. In game GUI testing, Liang et al. [137] proposed AG3, combining intelligent exploration with image analysis to detect multilingual text overlap and overflow. AG3 integrates VGG16 feature extraction, YOLOX detection, ViT patch classification, and multilingual OCR, and uncovered over 2,000 real defects in deployment.

Motor-impairment users have long been understudied. Krishnavajjala et al. [25] proposed MotorEase, combining dynamic analysis, object detection, and semantic modeling to detect designs unfriendly to motor-impairment users (e.g., hard-to-identify close buttons, insufficient touch areas). MotorEase uses GloVe embeddings and Faster-RCNN along with UI metadata and screenshots generated by existing tools, requiring no test scripts. It achieves about 90% average accuracy and under 9% false positives, outperforming AS [36] and Groundhog [128], filling a gap in motor-accessibility tooling.

For low-vision users, Zhang et al. [138] proposed ALVIN, constructing GUI graphs and applying GCN-based node classification to identify inaccessible components. By avoiding redundancy from GUI hierarchies, ALVIN improves adaptation for low-vision contexts. Experiments on 48 mobile apps show 83.5% precision and 78.9% recall, and ALVIN can model inter-component relations to avoid invisible redundancy and mishandling of small deviations caused by rule limitations.

5.5.3 Applications of LLMs in Accessibility Detection

The introduction of LLMs adds semantic reasoning and human-experience simulation to accessibility testing. Maryam et al. [139] proposed AXNav, a multi-agent testing system powered by GPT-4. AXNav receives natural-language instructions, generates test plans, and executes tests by simulating AT user operations, supporting checks for VoiceOver, large text, bold text, etc., and outputting annotated testing videos. AXNav substantially increases automation and is regarded as useful by testers. However, it is not designed to comprehensively scan all issues; rather, it marks a subset of issues during replay to assist manual testing. Its navigation replay may not fully match real user behavior, and it can fail in some cases (e.g., deciding when to stop or handling off-screen elements requiring scrolling).

Rule-based automated detectors have limited coverage on screen-reader-specific errors [59][140], and their feedback is often insufficiently detailed or explainable, reducing developers' motivation and ability to improve mobile apps. Zhong et al. [52] developed ScreenAudit, focusing on screen-reader interaction.

By recording TalkBack outputs and using LLMs to analyze their semantics, ScreenAudit automatically identifies semantic errors and accessibility issues, offering richer context explanations and repair suggestions. Its coverage is substantially higher than traditional tools: average coverage reaches 69.2%, far above AS [36] and Accessibility Inspector [35] at 31.3%. ScreenAudit can discover issues initially missed by experts and provide concrete repair suggestions. Its limitation lies in reliance on TalkBack-focusable elements, which may cause it to miss operable but non-focusable elements.

Overall, with the introduction of CV models and deep learning, perceivability-oriented issues such as insufficient spacing, abnormal button sizing, and low text-background contrast can now be effectively detected. The detection paradigm is evolving from the perceivable layer toward a perceivable-semantic co-evolution, shifting from “detecting whether issues exist” to “understanding issue essence and predicting UX impact,” and reshaping the intelligent landscape of accessibility detection. In terms of coverage, existing research can collectively detect 26 of 33 typical accessibility problems (78.8%), yet each single study covers relatively few (usually 1-9), resulting in a fragmented tool distribution. Fragmentation increases developers’ burden to combine tools and may produce redundancy, duplicate reporting, or omissions, limiting practical value. More importantly, although detection progress is substantial, mainstream work still focuses on identification, with insufficient attention to effective feedback, optimization, and repair driven by detection results; systematic post-processing and integrated toolchain support remain lacking. Therefore, the next section turns to mobile accessibility repair techniques, summarizing key strategies and exploring their linkage with detection mechanisms.

6 ANSWER TO RQ3: HOW CAN THE LANDSCAPE OF MOBILE APPLICATION ACCESSIBILITY REPAIR TECHNIQUES BE CATEGORIZED, AND WHAT ARE THEIR REPAIR PIPELINES, GENERALIZABILITY, AND FUNCTIONAL PERFORMANCE?

Based on a systematic analysis of 17 representative repair studies, this section summarizes the landscape and deployment pathways of mobile application accessibility repair. We construct a three-category framework—rule-driven and optimization-based repair, ML-based repair, and LLM-assisted repair—and compare them along repair pipelines, generality, and strengths/limitations. We then conclude with an overall assessment of the state of mobile accessibility repair research.

6.1 Rule-Driven and Optimization-Based Repair

This line of work treats accessibility repair as a problem of formal modeling and search/optimization. It relies on manually designed rules, heuristic algorithms, or code mutation strategies to automatically generate repair patches, and is commonly used in early research prototypes and lightweight integration scenarios. The core idea is to satisfy accessibility guidelines while preserving the original layout, visual aesthetics, and functional integrity as much as possible, yielding high interpretability and strong engineering fit.

Typically, these methods formulate repair as a multi-objective optimization problem. By automatically adjusting UI attributes (e.g., size, color, position), they seek to meet accessibility constraints while retaining the original layout structure and visual style. This modeling naturally handles multiple—often conflicting—objectives, such as “increasing contrast” vs. “preserving the original palette” or “enlarging widgets” vs. “avoiding layout disruption.” To solve such problems, meta-heuristic search is widely adopted, with GA being the most common choice. GA iteratively produces high-quality candidate repairs through selection, crossover, and mutation. Combined with MOO, these approaches output a Pareto frontier representing different trade-offs among objectives. Several systems (e.g., SALEM, ScaleFix) follow this general recipe; the overall pipeline is shown in Fig. 11.

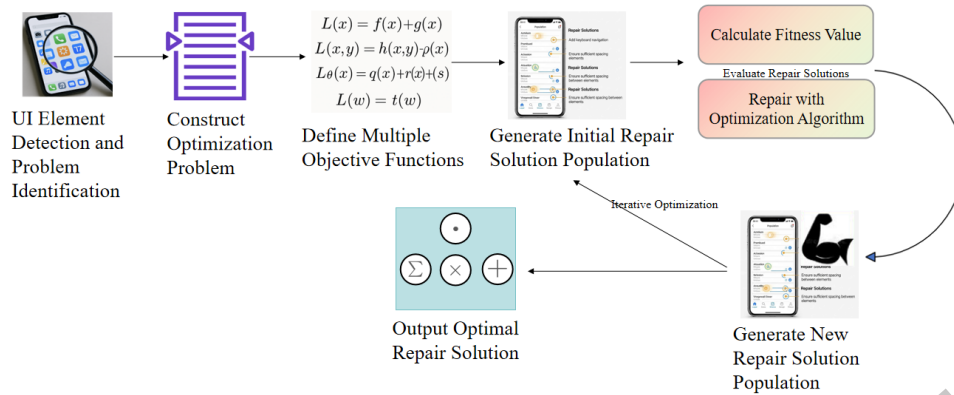


Fig. 11 Optimization-based repair framework

1) SALEM: Size-Based Multi-Objective Optimization Repair

SALEM (Size-based inAccessibility repair in Mobile apps), proposed by Alotaibi et al. [141], is the first automatic repair system focusing on size-related accessibility issues in GUIs. Based on genetic algorithms and multi-objective optimization strategies, this method enhances accessibility while maximizing interface consistency. SALEM's repair process consists of three stages:

- **Element Identification:** Grouping UI elements by their logical positions (XPath), sizes, and category types using the DBSCAN clustering algorithm to ensure visual consistency among components during repair.
- **Structure Modeling:** Introducing a "Size Relation Graph (SRG)" to characterize size dependencies between components, identifying the minimum necessary adjustment elements and attribute subsets to avoid unnecessary changes to the entire UI.
- **Solution Search:** Generating repair patches via a multi-objective genetic algorithm, with a fitness function that penalizes layout distortion, alignment disruption, or spacing compression caused by repairs. The system can automatically compile the repaired APK file, successfully repairing approximately 99% of size issues.

Despite its effectiveness, SALEM suffers from excessively long compilation times (accounting for ~98% of total runtime) and only focuses on size issues, potentially introducing new problems (e.g., abnormal component spacing) in certain scenarios [22].

2) AGAA: Genetic Algorithm Repair for Undersized Text and Information Redundancy

Xu et al. [142] proposed the AGAA method, expanding the application scope of genetic algorithms to address visual impairment users' difficulties in perceiving small fonts and redundant information. AGAA introduces a GUI repair framework based on graph modeling, comprising three key technologies:

- **Component Relation Graph (CRG) Construction:** Integrating XML structure and visual rendering information to model the GUI as a graph structure, including structural edges (parent-child relationships) and dependency edges (style and logical associations).
- **Issue Capture:** Identifying undersized text by comparing component sizes against Google's accessibility guidelines, while quantifying component clickability and layout occupancy to uncover information redundancy.
- **Solution Generation and Consistency Maintenance:** Optimizing repair strategies using genetic algorithms and maintaining consistency among similar components during size adjustment via DBSCAN clustering to maximize design aesthetics.

AGAA can directly modify source code and automatically deploy repair patches, demonstrating good performance in practical applications. However, it relies on component identification information, making it difficult to handle WebView components or non-standard designs, thus limiting its applicability.

3) ScaleFix: Multi-Objective Optimization Repair for UI Scaling Accessibility Failures

To address UI failure issues caused by assistive functions (e.g., scaling services), Alotaibi et al. [143] proposed the ScaleFix system, focusing on repairing UI Scaling Accessibility Failures (USAFs). This method constructs a UI Relation Graph (UIG) to model size, weight, and spatial constraint relationships between UI elements, thereby narrowing the repair search space. ScaleFix's fitness function integrates five objectives: minimizing text truncation, element missing, and element collision issues, maximizing the preservation of original positional relationships, and reducing repair interference. It achieves a repair success rate of 90% and significantly improves UI accessibility by 23%. The repaired

UI approaches manual optimization in aesthetics and readability. However, it has weak support for WebView, AdView, and code-defined UI elements.

4) Iris: Context-Aware Color Contrast Repair

Color contrast issues are common barriers in mobile accessibility [144]. Zhang et al. [24] proposed Iris, an automated repair method that uses context-aware strategies to select alternative color schemes for insufficient text and image contrast. Iris's key strategies include:

- Reference Database Construction: Building a reference database with numerous color-compliant UI components, categorized by component types.
- Alternative Color Selection: Prioritizing color matching from similar components or the same application, maintaining hue and saturation in the HSV model to avoid incongruity.
- Repair Implementation: Precisely locating problematic components via static data flow analysis and ensuring page coordination using Gestalt psychology's harmonic principles.

Iris successfully repairs 91.38% of contrast issues without compromising design consistency, effectively improving user satisfaction and receiving positive developer feedback. However, the current version has limited support for components without IDs, complex components like WebView, and compatibility with third-party libraries and new UI frameworks (e.g., Jetpack).

5) RGNF: Navigation Flow Reconstruction for Visually Impaired Users

Regarding GUI navigation structure accessibility, Zhang et al. proposed the RGNF system [145], which enhances the usage consistency for visually impaired users by reconstructing the GUI Navigation Flow (GNF). Based on user cognitive models, the system groups UI components using Gestalt psychology's proximity and similarity principles and redraws navigation orders via the DFS strategy to ensure components are read aloud consecutively in the expected order. Experimental results show that this method achieves a similarity of 0.921 with users' ideal navigation flows, and accessibility metrics improve to 90.31%. Limitations include incapability of handling GUIs with non-focusable components, unsuitability for pure text applications, and compatibility issues with some in-app reading tools.

6.2 Learning-Based Repair Methods

Learning-based repair methods leverage deep learning models to learn accessibility repair patterns from historical data, possessing certain generalization capabilities to automatically recommend or generate repair suggestions. They represent an important development direction in accessibility repair research in recent years. Such methods typically rely on neural architectures like convolutional neural networks (CNNs), recurrent neural networks (RNNs), and Transformers to learn complex visual-semantic relationships from large volumes of UI elements, screen screenshots, and their associated accessibility attributes or label data. This enables them to complete repair tasks such as generating missing labels, identifying UI elements, and semantic complementation. Excelling in processing unstructured data, these methods feature strong semantic reasoning and cross-interface adaptation capabilities, having demonstrated significant effectiveness in practical applications. Their typical system framework is shown in Figure 12.

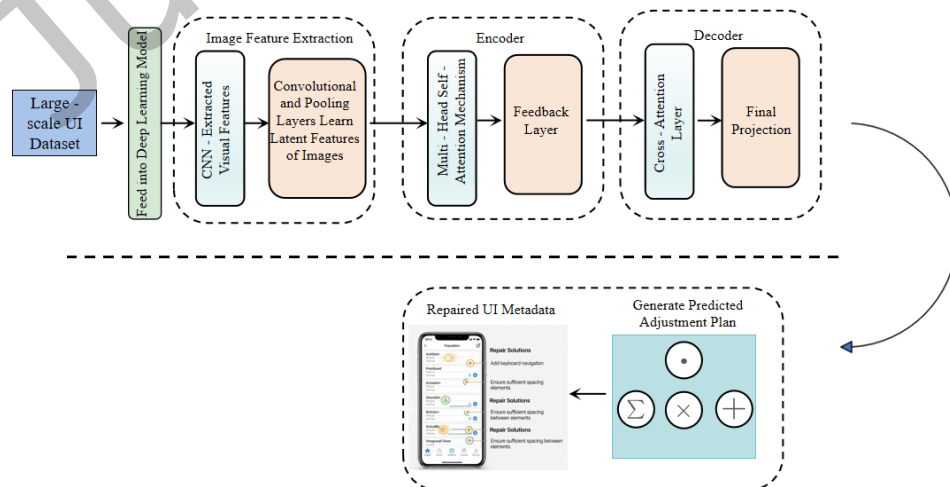


Fig. 12 Deep-learning-based repair framework

The LabelDroid system proposed by Chen et al. [146] was among the first to use deep learning for automatically generating UI image button labels. It models label prediction as an image captioning task, using a CNN to extract image features and a Transformer encoder-decoder structure to generate natural language labels word by word. Experiments show that LabelDroid's generated labels surpass manual annotations in semantic quality, but its performance in real-world scenarios with missing labels is limited due to data imbalance in the training set and the lack of consideration for icon contextual information.

To expand LabelDroid's applicability [147][148][149], Li et al. [150] further proposed the Widget Captioning task, introducing multi-modal inputs (UI structure + screen images). By fusing structural and visual features through a Transformer and ResNet, this method generates UI element descriptions with structural awareness, significantly improving the contextual relevance and semantic accuracy of labels. However, it still faces confusion with adjacent elements.

To address the insufficient utilization of contextual information in the above methods, Mehralian et al. [75] proposed the Coala system, which for the first time introduces icon usage contexts (e.g., mobile app type, activity name, UI hierarchy relationships) for auxiliary generation. Combining an image encoder (ResNet) and a context encoder (LSTM), Coala achieves higher generation accuracy than LabelDroid, especially for non-predefined labels. However, it may still generate incorrect labels for long-tail samples or in contexts with insufficient information.

Aiming to break free from dependence on UI metadata, Chen et al. [87] proposed the Pixel-Only method, which performs icon classification and label generation based solely on pixel images. Through few-shot learning and heuristic semantic completion mechanisms, this method demonstrates new potential in platform adaptability and generalization capabilities, but it still needs verification for cross-platform compatibility and symbolic polysemy recognition.

To tackle the challenges of data imbalance and label generation quality, Ma et al. [151] designed the Metila system, constructing a dual-stream Transformer model based on the Mean Teacher Learning mechanism. It significantly outperforms previous systems in multiple metrics such as BLEU and accuracy. This method ensures both training stability and long-tail sample learning capabilities, enhancing overall semantic quality, though the generated results still slightly lag behind manual annotations.

In terms of practical repair capabilities, Zhang et al. [22] proposed the AccessFixer system, innovatively using a Relational Graph Convolutional Network (R-GCN) to collaboratively adjust attributes such as size, spacing, and color contrast, addressing three common categories of accessibility issues in GUIs. By modeling component attributes through Fourier spectral signals and achieving local consistency repair on the GUI graph, it successfully repairs over 80% of issues, significantly outperforming traditional methods that only handle sizes. However, it has blind spots in processing text-free components and elements with missing attributes.

Additionally, the Screen Recognition method [77] demonstrates how to automatically generate accessibility metadata by combining visual models and heuristic rules, greatly expanding the scope of assistive function support. Through multi-stage detection, text recognition, and semantic reasoning, this method has achieved significant improvements in user studies, highlighting the enormous potential of deep learning in interface semantic understanding [80][121][152].

Overall, learning-based repair methods exhibit powerful capabilities in handling complex and diverse UI scenarios, particularly suitable for tasks such as label generation, semantic completion, and attribute prediction. However, they rely on large-scale high-quality datasets and still face challenges such as result interpretability and cross-application consistency. Future research needs to further enhance their generalization and controllability, combined with rule guidance mechanisms to strengthen system stability and deployment practicality.

6.3 Large Model-Assisted Repair Methods

In recent years, with the continuous evolution of LLMs, their remarkable advantages in cross-modal understanding, contextual awareness, and code generation have propelled mobile application accessibility repair into a new intelligent phase. Compared with traditional rule-based or deep learning approaches, LLMs possess capabilities such as multi-turn interaction, few-shot learning, and task generalization, emerging as a critical foundation for constructing "Intelligent Repair Agents".

Such methods typically take GPT-like LLMs as the core, generating repair suggestions—including alternative text, hint text, and source code repairs—through natural language prompting, contextual modeling, and multi-modal input fusion. Their workflow covers stages such as problem understanding, context extraction, prompt construction, and repair suggestion generation, with some methods also introducing feedback mechanisms to further optimize suggestion quality. Figure 13 illustrates the framework of typical LLM-based repair methods.

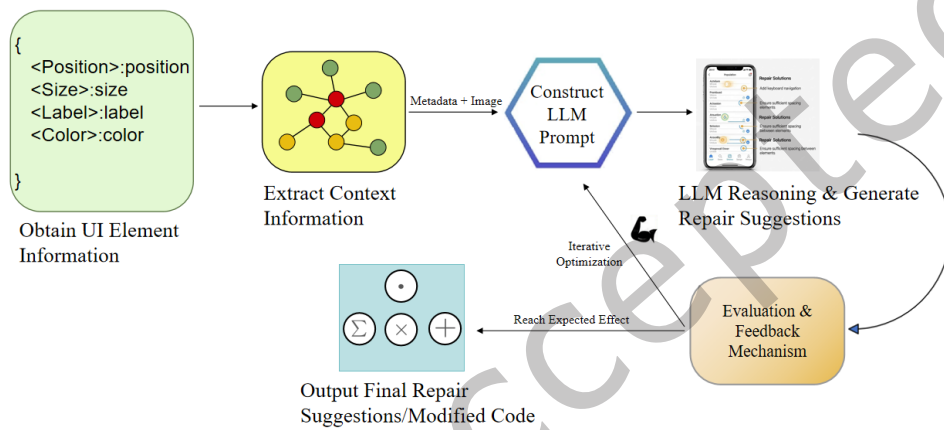


Fig. 13 Framework of typical LLM-based repair methods

Compared with traditional deep learning methods (such as LabelDroid [146] and Coala [75]), the repair methods based on LLMs have stronger semantic reasoning and context-aware capabilities, can effectively process complex unstructured data, and still maintain high performance in environments with scarce data and incomplete context. To solve the problem of missing alternative text for image components in mobile applications, Haque et al. [153][154] proposed the IconDesc method, which for the first time uses large multi-modal models (such as Gemini) and LLMs to jointly realize the automatic generation of icon alternative text. Its core process includes: extracting visual features from icon pixels and mapping them to icon labels; extracting text information in the icon combined with OCR; extracting icon context from the DOM tree (such as class name, resource ID, parent and sibling node content); fusing the above information to construct prompts and input them into LLMs to generate the final alternative text. IconDesc is superior to existing methods such as LabelDroid, Coala, Pix2Struct [155] and PaliGemma [156] in many evaluation indicators such as CIDEr and SPICE, showing strong performance under the conditions of limited data and partial UI screens. Nevertheless, problems such as misrecognition, improper handling of small icons, and DOM context misleading still affect its generation quality.

Aiming at the insufficient attention to the hint text problem of text input components in existing research, Liu et al. [157] proposed the HintDroid method, which for the first time uses LLMs to realize the automatic generation of hint text based on the context of the GUI page. This method designs a multi-level prompt strategy including GUI entity extraction, example enhancement and feedback mechanism, and significantly improves the relevance and diversity of text hints by combining situational learning and GUI analysis. Experiments show that HintDroid has improved the exact match rate by more than 82%, and has shown strong adaptability in different application categories. However, this method relies on view

hierarchy data, and as an offline processing method, it needs to repackage the application, so its application scope is limited.

In addition, existing accessibility detection studies (such as AS [36], Groundhog [128], etc.) can identify issues, but they often lack guidance on how to solve these problems. Single-point repair techniques (such as SALEM [141], Iris [24], ScaleFix [143], etc.) can only address a small part of the issues. Mehralian et al. [158] proposed the FixAlly technology, which for the first time introduces the "Plan-Localize-Fix" architecture. This method uses multi-agent LLMs to simulate the developer repair process: the Planner first generates natural-language repair strategies based on problem descriptions, then the Localizer locates the source code positions, and finally the Fixer outputs repair code snippets. FixAlly can handle various accessibility issue types (such as too-small click areas, insufficient color contrast, etc.) and provides diversified repair options. Empirical evaluations show that the acceptance rate of its repair suggestions reaches 69.4%. However, FixAlly currently only supports the iOS platform, handles only one issue at a time, and still struggles to support cross-component linkage repairs.

To sum up, the LLM-based repair method breaks through the traditional method's dependence on data, structure and rules, and provides a new paradigm for the accessibility repair of mobile applications [159]. Its key advantages are: 1) it can integrate multi-modal information to understand UI semantics; 2) it supports context-aware and multi-round dialog repair; 3) it has cross-task migration and generation capabilities. Although it still faces challenges such as generalization ability, input limitations and platform adaptation, relevant studies have fully shown the development potential and practical prospects of LLMs in the field of accessibility repair.

7 CURRENT CHALLENGES AND FUTURE RESEARCH DIRECTIONS

The rapid development of LLM- and AI-enabled techniques is pushing SE tasks toward more intelligent and efficient directions [160]. However, constrained by the complexity of mobile app ecosystems and the boundaries of existing techniques, achieving comprehensive and universal mobile application accessibility still faces systemic challenges. This section summarizes current limitations around two core stages—issue detection and issue repair—and then discusses feasible extensions and future opportunities from both the SE lifecycle perspective and the AI-enablement perspective.

7.1 Challenges in Mobile Accessibility Detection

Mobile application accessibility detection methods play a critical role in supporting the accessibility-by-design lifecycle and improving the efficiency of systematic assessments [45]. Existing studies have evolved from early manual evaluations to static/dynamic testing, automated GUI exploration, AT-interaction simulation, and deep learning/LLM-driven semantic detection, achieving progress in speed, repeatability, and coverage. Nevertheless, the overall landscape still suffers from the following limitations:

(1) Fragmented detection capabilities, with insufficient coverage by individual tools. Although existing methods collectively can identify 28 out of 33 common accessibility issue types (78.8% coverage), a single study typically covers only 1-9 types. Such fragmentation forces developers to combine multiple tools, which often leads to inconsistent results, redundant reports, and coexisting misses, substantially weakening the practical value of detection in real-world engineering. It also implies a risk that the reported overall coverage may be “inflated” in practice.

(2) Detection precision remains inadequate, with widespread false negatives and false positives [18][89][132][138][161], especially in complex interactions or context-dependent scenarios where methods struggle to fully interpret context or assess content quality. For instance, static tools (e.g., Lint [104]) cannot detect dynamic properties that only manifest at runtime, easily creating a false sense of assurance; manual tools (e.g., AS [36]) can capture runtime issues, but are labor-intensive, non-scalable, lack cross-screen memory, and often generate reports with many duplicates, reducing efficiency; automated GUI exploration tools (e.g., MATE [56]) are limited by preset rules and often ignore how users with disabilities actually interact with mobile apps via AT; AT behavior simulation methods (e.g., Latte

[50]) better approximate real user experience, but their effectiveness depends on existing test cases or linear navigation, remains brittle, supports limited complex gestures, and still produces false positives/negatives; deep learning approaches (e.g., OwlEyes [126]) sometimes show low precision in localizing problematic regions, and the realism of synthesized training data is debatable; even advanced LLM-based methods (e.g., ScreenAudit [52]) may rely on TalkBack-focusable elements, have limited contextual understanding, and thus miss elements that are operable but not focusable. Therefore, improving precision in complex and dynamic scenarios remains the primary bottleneck.

(3) Lack of post-processing mechanisms, making it hard to advance repair. Most studies focus on finding issues but provide little in terms of executable repair suggestions, prioritization, or automated repair interfaces. This creates a gap between detection and repair. When detection outputs cannot be directly translated into actionable engineering tasks, their improvement impact is significantly diluted.

(4) Non-unified detection reports hinder downstream repair. Different tools generate reports with inconsistent formats, granularity, and terminology, and often lack cross-screen aggregation and global issue-pattern mining capabilities [51]. For example, manual tools usually report results screen by screen without “memory across scans.” Huq et al. [17] show that such manual reports may contain many duplicates. As a result, developers struggle to track, govern, and regress accessibility defects systematically across the lifecycle, which in turn harms repair efficiency and quality [162][163].

To address these limitations, future work should focus on the following directions to build more universal, efficient, and scalable accessibility solutions:

(1) Establish unified mobile accessibility standards and report specifications, providing consistent detection-repair foundations [51][164]. Future research needs more comprehensive and fine-grained standards tailored to mobile characteristics, along with uniform reporting norms that supply detailed, explainable issue descriptions and repair guidance to facilitate developer adoption. This can remove the burden caused by fragmented standards and improve efficiency.

(2) Improve coverage and precision. 1) More intelligent GUI exploration: To overcome limited coverage on screens requiring specific inputs or involving sensitive data, future exploration strategies can incorporate RL or symbolic execution to reach deeper mobile app states more effectively [53][165][166], ensuring tools traverse all functional paths. 2) Comprehensive AT behavior simulation: Current simulations poorly support complex gestures and diverse assistive services. Future methods should simulate richer assistive services and complex interactions (e.g., drag, pinch, multi-finger gestures) to reflect real disability-use scenarios more faithfully. 3) Context understanding and semantic analysis: Although deep learning and LLMs are already used, they still cannot fully interpret context or assess content quality. Future models must enhance semantic understanding to detect subtle errors such as reading order or contextual semantics of elements, reducing reliance on human judgment [167][168]. 4) Handling dynamic content and complex UIs: Dynamic loading, navigation loops, or highly customized UI components cause misses and spurious alarms. More robust detection is needed to accurately capture and analyze such scenarios [70].

(3) Build end-to-end, full-lifecycle detection-repair pipelines integrated into CI/CD for continuous assurance. 1) Deep fusion of detection and repair: Given current fragmentation [24][28], future solutions should translate detection outputs into actionable repair suggestions, or even automate repair, reducing developer workload and improving efficiency. 2) CI/CD integration: Embedding accessibility detection and repair into CI/CD enables automated, continuous accessibility assurance, ensuring issues are discovered and addressed at every commit or release [169][170].

(4) Introduce user capability-difference modeling for targeted detection and personalized adaptation. Existing standards/tools are largely static and cannot dynamically adapt UIs to individual users. Future work should develop capability models that support detection and repair based on disability type, preferences, and usage habits, enabling true “accessibility-by-default” with on-demand adaptation [171].

7.2 Challenges in mobile application accessibility repair

Mobile accessibility research is shifting from “detection-centric” to “proactive repair.” Existing repair methods include optimization-based, deep learning-based, and LLM-enabled automation, showing early success for issues such as label completion and size/contrast adjustment. Yet, several challenges remain:

(1) Insufficient issue coverage is the central challenge. Of 33 issue categories, current repair studies address only 15 (45.45% coverage), and a single study typically covers only 18.2%. This reveals both the current focus and the large number of underexplored issue types. Repair coverage is far below detection coverage (45.45% vs. 78.8%), highlighting a structural imbalance: many detected issues lack automatic or semi-automatic repair, leaving developers to manually fix most problems—costly, slow, and error-prone. Thus, research must shift toward robust and comprehensive repair mechanisms, especially for still-unaddressed issue types.

(2) Most approaches target a single issue or specific component attributes, with limited applicability to cross-screen flows, complex gestures, dynamic UIs, closed-source apps, and cross-file modifications. This makes them insufficient for the composite and systemic nature of mobile accessibility. A unified, comprehensive repair framework is urgently needed.

(3) Repair precision and generalization remain limited. Some methods still mis-handle complex/dynamic UIs, producing misses or spurious repairs. They also over-rely on datasets; when icons are rare, error rates rise substantially [75].

(4) Insufficient contextual and semantic understanding is a key bottleneck. Many methods cannot fully interpret context or evaluate content quality [77][146], and often miss subtle defects requiring semantic judgment, such as AT reading order or the contextual meaning of elements.

(5) Lack of unified evaluation criteria and report formats prevents effective comparison across studies and forces developers to reconcile inconsistent tools. Overall, repair research remains weak in post-processing and in advancing accessibility improvement based on detection outputs [24].

Future repair research can move forward along these directions:

(1) Expand the coverage of repair targets. With only 15/33 categories repaired so far, future work should prioritize issues affecting cognitive-impaired and multi-disability users, and develop methods for complex interactions (e.g., drag, pinch) and dynamic UI changes (e.g., looping lists, disappearing menus). As wearables, VR/AR, and AI-driven interaction proliferate, accessibility repair for these emerging modalities also becomes crucial [172].

(2) Increase repair intelligence and automation. 1) Multimodal fusion: Explore deep learning and LLMs for jointly leveraging image, text, structure, and behavior data, enabling deeper UI semantics and intent understanding, hence more precise repair suggestions. 2) Context-aware repair: Use UI context (e.g., user habits, scenarios, device configurations) to support personalized, adaptive optimization. 3) End-to-end automated repair: Develop pipelines from detection to code-level repair with minimal human intervention, capable of handling interdependent issues and cross-file changes. 4) RL for repair: Introduce RL to learn and optimize repair strategies through interaction with mobile app environments, adapting to complex UIs and user needs.

(3) Strengthen evaluation and validation. Build comprehensive and objective metrics that consider not only accuracy but also user experience, UI aesthetics, performance, and security impacts. Incorporate real-user A/B testing and longitudinal studies to validate real-world effectiveness and durability. Simulation environments that preview repaired UIs can further improve developer efficiency and repair quality.

(4) Promote interdisciplinary collaboration and standards unification. Encourage collaboration among human factors, SE, cognitive science, and HCI to integrate multi-disciplinary theories into repair methodologies. Meanwhile, establish finer-grained and more universal mobile accessibility standards, especially considering mobile-specific interactions and emerging technologies. Cross-platform repair solutions can reduce repeated adaptation costs.

(5) Improve toolchain integration and usability. Develop repair tools with higher integration and more developer-friendly interfaces, embedding them into IDEs and CI/CD. Provide detailed and explainable

repair reports (issue descriptions, suggestions, code examples, visual previews). Open repair datasets and benchmarks will also foster community collaboration and sharing.

7.3 SE perspective: addressing accessibility challenges across lifecycle stages

To situate our findings within SE, we clarify how accessibility issues relate to lifecycle activities and how SE processes can be extended to better support accessibility. Fragmentation and poor deployability of detection/repair are not only technical problems but also reflect a structural gap where accessibility is not stably embedded in the SE lifecycle. Thus, we discuss how to revise SE processes accordingly.

(1) Requirements engineering. Most mobile apps still treat accessibility as optional, causing late-stage patching and fragmented detection results. SE practice can be extended with accessibility-specific requirement templates, user capability role models, and acceptance criteria (e.g., AT navigation constraints, perceptual load thresholds). Elevating accessibility to a first-class requirement helps prevent downstream report inconsistency and coverage gaps.

(2) Design and modeling. Traditional UI design rarely semantically models user tasks or AT behaviors, leading to semantic limitations in detection. SE design tools should integrate accessibility-aware static checks, focus-chain visualization, and cross-device previews. MDE can represent accessibility rules in a platform-independent manner, alleviating cross-platform inconsistency.

(3) Implementation and toolchain integration. Detection fragmentation partly stems from the absence of unified IDE/build-time support. SE practice should introduce accessibility-aware static analyzers, enforceable code annotations, and compile-time constraint checkers. Integrating these into DevOps pipelines shifts accessibility checks from ad hoc usage to reliable, reproducible quality gates.

(4) Testing and validation. Existing methods struggle with runtime behaviors, gesture interaction, and dynamic AT use. SE testing frameworks need dedicated AT simulators, multimodal gesture engines, and semantic test oracles beyond structural metrics. Cross-screen memory and multi-batch consistency checks can reduce false positives and improve engineering usability.

(5) Maintenance and evolution. Mobile UIs evolve rapidly, causing rule and ML-model drift. SE configuration management should add accessibility-aware version control, automated accessibility regression testing, and logs about repair impacts across versions. LLM-based repair agents must meet reproducibility requirements—pattern-constrained outputs, verification logs, and traceable patch artifacts—aligning with broader trustworthy AI-for-SE research.

Overall, embedding accessibility throughout the SE lifecycle is the key to transforming today’s tool-centric, fragmented approaches into stable, industrializable practices.

7.4 AI enablement: making accessibility a broader reality

AI can not only improve detection/repair accuracy but also transform accessibility from “after-the-fact remediation” into “proactive, continuous, user-centered” engineering, thus enabling real-scale adoption.

First, LLMs and multimodal models can semantically understand UI layout, user intent, perceptual load, and AT behaviors, uncovering issues beyond rule violations, such as cognitive overload or gesture conflicts.

Second, AI enables scalable personalization: by modeling user capability profiles (e.g., vision level, motor patterns, cognitive load), systems can adapt UIs in real time, shifting from a “universal design” paradigm to an “accessibility-by-default, adaptive-on-demand” paradigm.

Third, AI-assisted development tools—accessibility-aware copilots, code-generation agents, and design-evaluation models—can embed accessibility early in requirements, prototypes, and UI design, reducing downstream detection/repair burdens.

Fourth, integrating AI into CI/CD enables continuous accessibility regression testing across devices, themes, scaling factors, and user groups, realizing “continuous accessibility.”

However, dataset bias, limited AT simulation, missing semantic modeling, and the lack of verification standards for AI-generated patches still restrict effectiveness. Breaking these limits requires extending SE methods, reshaping processes, and incorporating accessibility into SE education so developers treat it as a first-class requirement rather than a downstream patch.

A key takeaway is that accessibility must increasingly “shift left” –embedded in requirements, prototyping, and design rather than relying on post-hoc detection/repair. AI is decisive in this shift-left process. LLMs and multimodal models can evaluate wireframes and early prototypes, generate accessibility-aware design alternatives, predict AT interaction conflicts, and automatically produce accessibility user stories, capability roles, and acceptance criteria. These early interventions greatly reduce the flow of violations into later lifecycle stages.

Moreover, AI assistants integrated into IDEs can provide real-time, context-sensitive accessibility guidance during coding, preventing many issues that would otherwise require downstream tools. In this sense, AI reduces—rather than increases—dependency on post-hoc tooling. Achieving this shift-left requires SE practice to adapt: requirements must include AI-generated capability models; design tools must open accessibility-aware feedback loops; CI/CD must support early accessibility gates; and SE education must train developers to realize “accessibility-by-design.”

Taken together, these mechanisms explain how AI can make accessibility a broader reality while reducing the cost of late-stage detection and repair. The core insight of this survey is that accessibility must be treated as a requirement as fundamental as project functionality, and embedded from the earliest stages. Doing so substantially reduces reliance on costly, fragile, and limited post-hoc tools, enabling a development process centered on prevention rather than discovery.

8 LIMITATIONS AND THREATS TO VALIDITY

8.1 Literature Acquisition and Screening

Although we designed a multi-stage search process with venue tiering, terminology expansion, multi-database batch searches, and forward/backward snowballing, and systematically queried nine databases (ACM DL, IEEE Xplore, ScienceDirect, SpringerLink, Wiley, Web of Science, Scopus, CNKI, and arXiv), the completeness of our literature coverage still cannot be fully guaranteed. (1) The search strings were manually constructed and iteratively refined based on a domain-specific terminology database, so we may still have missed studies that use non-standard terminology (e.g., domain-specific descriptions of disabilities or idiosyncratic ways of describing accessibility) or that were published in venues outside our target list. Therefore, this review should be regarded as a representative view of high-quality and traceable research, rather than an exhaustive enumeration of all existing work. (2) Some HCI-oriented studies that only marginally involve mobile accessibility may have been excluded during quality assessment. At the same time, the application of inclusion/exclusion criteria inevitably involves subjective judgment. To mitigate this, we adopted a dual-reviewer independent screening process and resolved disagreements through discussion, in order to reduce the influence of individual subjectivity as much as possible. (3) Our search window ends on July 1, 2025. In rapidly evolving areas such as mobile accessibility and LLM-enabled software engineering, new tools, frameworks, and practices are constantly emerging. As a result, our synthesis of detection and repair technique landscapes reflects a time-bounded snapshot rather than long-term stable conclusions. When using this review as an evidence baseline, subsequent research should partially update and adjust the conclusions in light of newer literature and platform changes, especially in the context of improved LLM capabilities and evolving platform policies.

8.2 Limited Coverage of Industrial Practice and Grey Literature

In addition to the SLR itself, we selected several representative tools and industrial practices as auxiliary analyses to complement the SLR evidence and strengthen the engineering orientation of our work. However, for reasons of reproducibility and argumentative rigor, we can only systematically cover cases

whose public documentation is relatively complete or that are described in sufficient detail in academic papers. A large number of closed-source enterprise-level detection/repair pipelines and region-specific regulatory practices are difficult to incorporate into a unified analysis framework. Consequently, our characterization of the current industrial landscape is inevitably incomplete. Nevertheless, we argue that the mainstream tools, typical industrial practices, and related academic studies selected in this review are sufficiently representative to outline the core technical pathways and engineering application trends in mobile accessibility detection and repair, and to provide a valuable reference baseline for subsequent research and practice.

8.3 Task and Platform Adaptability

Our study focuses on native mobile applications and builds an accessibility issue taxonomy, an evolution path of detection techniques, and a repair-technique taxonomy around three main threads: issues, detection, and repair. These frameworks are grounded in the POUR principles of WCAG 2.x, user-ability models, and the software engineering lifecycle, and emphasize alignment with platform Assistive Technology (AT) behavior and engineering practice. Inevitably, however, they also reflect our modeling perspective. For example, we place greater emphasis on issues related to visual and motor abilities, while the coverage of cognitive and auditory impairments and multi-disability scenarios is comparatively limited. This is partly due to the relative scarcity of related studies in the existing literature, and partly because accessibility specifications and tools for visual/motor impairments are more mature on native mobile platforms. We also do not provide a systematic survey of Web applications, wearables, XR, or multi-device collaborative interaction. In practice, mobile and Web applications differ fundamentally in terms of technical architecture, interaction modalities, and system integration, and the research community typically treats them as separate targets rather than merging them.

Therefore, the taxonomies and terminology in this paper are better understood as a WCAG-compatible, mobile-oriented software engineering baseline, rather than the only possible theoretical partition of the field; different researchers may reasonably adopt slightly different boundaries and levels of granularity. How to extend accessibility engineering to additional platforms, richer ability models, and multi-modal interaction patterns remains an important direction for future work.

9 CONCLUSION

We conducted a SLR on mobile application accessibility, grounded in authoritative databases and a reproducible search-screening-assessment workflow. We performed multi-round retrieval using credibility-tiered sources and standard terminology clusters across ACM DL, IEEE Xplore, ScienceDirect, SpringerLink, Wiley, Web of Science, Scopus, and CNKI (with arXiv as a seed source), covering studies up to 2025-07-01. We then executed deduplication, applied inclusion/exclusion criteria, conducted forward/backward snowballing, and performed quality assessment. In total, 76 papers were included. Based on these studies, we systematically synthesized accessibility interaction frameworks, relevant standards, the issue landscape, the evolution of detection paradigms, and repair techniques, and summarized common challenges across methodological, technical-mechanism, and system-implementation layers.

The field has shifted from rule-based compliance checking toward user-perception modeling and intelligent repair. Deep learning and recent LLMs have significantly enhanced automation, especially in generating prompts and alternative texts, code repair, and strategy recommendation, showing the potential of “intelligent repair agents.” Yet these approaches remain constrained by generalization, explainability, robustness, and industrial applicability.

Drawing on the existing literature and technical practice, mobile application accessibility research over the past decade has evolved from rule-based compliance detection toward user-perception modeling and intelligent repair, exhibiting a broader migration “from compliance verification to personalized support.” however, four core gaps persist:

- Insufficient semantic modeling: Current detection methods primarily rely on static rules and UI structural features, lacking integrated modeling of UI semantics, task flows, and user intent, resulting in frequent false positives/negatives and limited robustness.

- Weak repair mechanisms: Most studies focus on detection, while truly closed-loop “detect-localize-repair-verify” systems remain largely absent, especially without unified frameworks for multi-disability coordination or dynamic-interaction repair.
- Missing user capability-difference modeling: Existing methods are built on generic assumptions and do not capture the concrete interaction patterns and real usage burdens of groups such as visually impaired users, older adults, and users with cognitive impairments, preventing genuine adaptive-on-demand support.
- Disconnect between standards and engineering tools: Although WCAG-like standards have become increasingly fine-grained, tool deployment still faces insufficient adaptation, platform fragmentation, and inconsistent enforcement, undermining universality.

Overall, although mobile accessibility techniques have achieved substantial progress, there is still a long way to go before the vision of “accessibility by default” can be fully realized. To further bridge the gap between systematic coverage and engineering practicality in real-world development settings, future research needs to continue deepening several key directions. Specifically, the major avenues for mobile application accessibility research include: (1) semantics-enhanced detection and interaction modeling—leveraging semantic graphs and LLMs to enable contextual understanding and cross-screen aggregation, thereby improving report consolidation and strategy generation; (2) multimodal repair—building coupled models across vision, language, and code to strengthen generalization and execution efficiency; (3) user-difference awareness—achieving personalized detection and repair based on capability profiles and interaction features, operationalizing “accessibility by default, adaptive on demand”; and (4) standard-tool integration—unifying reporting specifications and constructing open datasets and evaluation benchmarks to support the transition from academic prototypes to industrial deployment.

ACKNOWLEDGMENTS

This work was supported by the National Social Science Fund of China (21BYY106), the Key Program of the National Natural Science Foundation of China (62036001), the Key Project of the National Language Commission (ZD1145-110), the Ministry of Education Project (25YJC740057), and the Research Project of the China Disabled Persons' Federation (25&ZZ016).

REFERENCES

- [1] Luo J, Wu W, Yang M. Mobile Internet: Terminal Devices, Networks and Service[J]. Chinese Journal of Computers, 2011, 34(11): 2029-2051 (in Chinese).
- [2] Ericsson. Ericsson Mobility Report. 2024. <https://www.ericsson.com/4adc9b/assets/local/press-releases/asia/2024/emr-november-2024-cn-v3.pdf> (in Chinese).
- [3] Zhou M, Li XQ, Wang Q, Li Q, Shen C, Zhou YT. Survey on Acoustic-sensing -based Authentication on Mobile Devices. Ruan Jian Xue Bao / Journal of Software, 2025, 36(5): 2229-2253 (in Chinese). <http://www.jos.org.cn/1000-9825/7181.htm>.
- [4] Liu H, Yu J, Song S, Zhang M. Review of Mobile Internet Information Accessibility Research[J]. Journal of Jilin University (Science Edition), 2025, 63(01): 124-138. DOI: 10.13413/j.cnki.jdxblxb.2024474 (in Chinese).
- [5] World Wide Web Consortium (W3C). Mobile Accessibility at W3C. 2025. <https://www.w3.org/WAI/standards-guidelines/mobile/>.
- [6] World Health organization. Disability. 2024. <https://www.who.int/news-room/fact-sheets/detail/disability-and-health>.
- [7] World Wide Web Consortium (W3C). WCAG 2 Overview. 2019. <https://www.w3.org/WAI/standards-guidelines/wcag/>.
- [8] CAICT, ARA. White Paper on Information Accessibility. 2022. <https://www.caict.ac.cn/kxyj/qwfb/bps/202205/P020220518510041281463.pdf> (in Chinese).
- [9] ISO. ISO/IEC 25010:2023: Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – Product quality model. 2023. <https://www.iso.org/standard/78176.html>.
- [10] Hinderks A, Mayo F J D, Thomaschewski J, Escalona M J. Approaches to manage the user experience process in Agile software development: A systematic literature review[J]. Information and Software Technology, 2022, 150: 106957.
- [11] Luo S, Gong R, Zhu S. User Experience Oriented Software Interface Design of Handheld Mobile Devices[J]. Journal of Computer-Aided Design & Computer Graphics, 2010, 22(6): 1033-1041 (in Chinese).
- [12] Hoehle H, Aljafari R, Venkatesh V. Leveraging Microsoft's mobile usability guidelines: Conceptualizing and developing scales for mobile application usability[J]. International Journal of Human-Computer Studies, 2016, 89: 35-53.
- [13] Jabangwe R, Edison H, Duc A N. Software engineering process models for mobile app development: A systematic literature review[J]. Journal of Systems and Software, 2018, 145: 98-111.

- [14] Inostroza R, Rusu C, Roncagliolo S, Rusu V, Collazos C A. Developing SMASH: A set of SMARTphone's uSability Heuristics[J]. *Computer Standards & Interfaces*, 2016, 43: 40-52.
- [15] Păsărelu C R, Andersson G, Dobrea A. Attention-deficit/hyperactivity disorder mobile apps: A systematic review[J]. *International journal of medical informatics*, 2020, 138: 104133.
- [16] J. Luca E, Ulyannikova Y. Towards a user-centred systematic review service: The transformative power of service design thinking[J]. *Journal of the Australian Library and Information Association*, 2020, 69(3): 357-374.
- [17] Huq S F, Alshayban A, He Z, Malek S.# a11ydev: Understanding contemporary software accessibility practices from twitter conversations[C]//*Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. 2023: 1-18.
- [18] Kashif L, Faisal C M N, Mahmood T. Automated Accessibility Evaluation of Mobile Applications on Initial Stages of Design and Development[J]. *Applied and Computational Engineering*, 2023, 2: 499-507.
- [19] Bi T, Xia X, Lo D, Grundy J C, Zimmermann T, Ford D. Accessibility in software practice: A practitioner's perspective[J]. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 2022, 31(4): 1-26.
- [20] Khalajzadeh H, Shahin M, Obie H O, Agrawal P, Grundy J. Supporting developers in addressing human-centric issues in mobile apps[J]. *IEEE Transactions on Software Engineering*, 2022, 49(4): 2149-2168.
- [21] Hadadi S, Sarsenbayeva Z, Kay J. Starting well on design for accessibility: analysis of W3C's 167 accessibility evaluation tools for the design phase[C]//*Proceedings of the 25th International ACM SIGACCESS Conference on Computers and Accessibility*. 2023: 1-7.
- [22] Zhang M, Liu H, Chen C., Gao G, Li H, Zhao J. AccessFixer: Enhancing GUI accessibility for low vision users with R-GCN model[J]. *IEEE transactions on software engineering*, 2023, 50(2): 173-189.
- [23] Chen S, Chen C, Fan L, Fan M, Zhan X, Liu Y. Accessible or not? an empirical investigation of android app accessibility[J]. *IEEE Transactions on Software Engineering*, 2021, 48(10): 3954-3968.
- [24] Zhang Y, Chen S, Fan L, Chen C, Li X. Automated and context-aware repair of color-related accessibility issues for android apps[C]//*Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 2023: 1255-1267.
- [25] Krishna Vajjala A, Mansur S M H, Jose J, Moran K. MotorEase: Automated detection of motor impairment accessibility issues in mobile app UIs[C]//*Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 2024: 1-13.
- [26] Yu S, Fang C, Tuo Z, Zhang Q, Chen C, Chen Z, Su Z. Vision-based mobile app gui testing: A survey[J]. *arXiv preprint arXiv:2310.13518*, 2023.
- [27] Zhang M, Liu H, Du C, Wang T, Li H, Huang P, Chen C. Distinguishing GUI Component States for Blind Users using Large Language Models[J]. *ACM Transactions on Software Engineering and Methodology*, 2025.
- [28] Nie L, Liu H, Sun J, Said K S, Hong S, Xue L, Wei Z, Zhao Y, Li M. SoK: Detection and Repair of Accessibility Issues[J]. *arXiv preprint arXiv:2411.19727*, 2024.
- [29] UNESCO. Recommendation on the Ethics of Artificial Intelligence. 2022. https://unesdoc.unesco.org/ark:/48223/pf0000381137_chi.
- [30] World Wide Web Consortium (W3C). Mobile Web Best Practices 1.0. 2008. <https://www.w3.org/TR/mobile-bp/>.
- [31] World Wide Web Consortium (W3C). Web Content Accessibility Guidelines (WCAG) 2.1. 2025. <https://www.w3.org/TR/WCAG21/>.
- [32] Google. Build accessible apps - Android Developers. 2019. <https://developer.android.com/guide/topics/ui/accessibility>.
- [33] Apple Inc. Accessibility - Apple Developer. 2018. <https://developer.apple.com/accessibility/>.
- [34] State Administration for Market Regulation, Standardization Administration of China. GB/T 37668-2019 Information technology-Internet content accessibility technical requirements and conformance testing. 2019. <http://c.gb688.cn/bzgk/gb/showGb?type=online&hcno=35ECC696805C1A67C93B74FB6D0D8EFB> (in Chinese).
- [35] Apple. Developer Tool : Accessibility Inspector. 2019. <https://developer.apple.com/documentation/accessibility/accessibility-inspector>.
- [36] Google. Play Accessibility Scanner. 2016. https://play.google.com/store/apps/details?id=com.google.android.apps.accessibility.auditor&hl=en_GB.
- [37] Matos M, Seixas Pereira L, Duarte C. Evaluation of the Accessibility of Mobile Applications: Current Approaches and Challenges[C]//*International Conference on Human-Computer Interaction*. Cham: Springer Nature Switzerland, 2023: 352-371.
- [38] Mascetti S, Ducci M, Cantù N, Pecis P, Ahmetovic D. Developing accessible mobile applications with cross-platform development frameworks[C]//*Proceedings of the 23rd International ACM SIGACCESS Conference on Computers and Accessibility*. 2021: 1-5.
- [39] Wang Y, Liu X, Arora C, Grundy J, Hoang T N. Understanding VR Accessibility Practices of VR Professionals[C]//*Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. 2025: 1-17.
- [40] Etxabe-Antia A, Beitia-Amendarain A, González de Heredia-López de Sabando A, Justel-Lozano D. Characterisation of accessibility guidelines for digital technologies[J]. *Universal Access in the Information Society*, 2025: 1-21.
- [41] Mohamad Y, Kouroupetroglou C, Kirisci P T. Future Challenges of User Modelling for Accessibility[C]//*Universal Access in Human-Computer Interaction. Design and Development Methods for Universal Access: 8th International Conference, UAHCI 2014, Held as Part of HCI International 2014, Heraklion, Crete, Greece, June 22-27, 2014, Proceedings, Part I* 8. Springer International Publishing, 2014: 270-278.
- [42] Morrison C, Grayson M, Marques R F, Massiceti D, Longden C, Wen L, Cutrell E. Understanding personalized accessibility through

teachable ai: designing and evaluating find my things for people who are blind or low vision[C]//Proceedings of the 25th International ACM SIGACCESS Conference on Computers and Accessibility. 2023: 1-12.

- [43] Polanco Jr A, Liu T. Multidisciplinary Framework for Creating the Next-Generation of Human-Centered Design Guidelines[J]. Human-Centered Design and User Experience, 2023, 114(114).
- [44] Acosta-Vargas P, Salvador-Acosta B, Salvador-Ullauri L, Villegas-Ch W, Gonzalez M. Accessibility in native mobile applications for users with disabilities: A scoping review[J]. Applied Sciences, 2021, 11(12): 5707.
- [45] Silva C, Eler M M, Fraser G. A survey on the tool support for the automatic evaluation of mobile accessibility[C]//Proceedings of the 8th International Conference on Software Development and Technologies for Enhancing Accessibility and Fighting Info-exclusion. 2018: 286-293.
- [46] Paz F, Pow-Sang J A. A systematic mapping review of usability evaluation methods for software development process[J]. International Journal of Software Engineering and Its Applications, 2016, 10(1): 165-178.
- [47] Brajnik G. Comparing accessibility evaluation tools: a method for tool effectiveness[J]. Universal access in the information society, 2004, 3: 252-263.
- [48] Rafi D M, Moses K R K, Petersen K, Mäntylä M V. Benefits and limitations of automated software testing: Systematic literature review and practitioner survey[C]//2012 7th international workshop on automation of software test (AST). IEEE, 2012: 36-42.
- [49] Seixas Pereira L, Matos M, Duarte C. Exploring mobile device accessibility: challenges, insights, and recommendations for evaluation methodologies[C]//Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems. 2024: 1-17.
- [50] Salehnamadi N, Alshayban A, Lin J W, Ahmed I, Branham S, Malek S. Latte: Use-case and assistive-service driven automated accessibility testing framework for android[C]//Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems. 2021: 1-11.
- [51] Swearngin A, Wu J, Zhang X, Gomez E, Coughenour J, Stukenborg R, Garg B, Hughes G, Hilliard A, Bigham J P, Nichols J. Towards automated accessibility report generation for mobile apps[J]. ACM Transactions on Computer-Human Interaction, 2024, 31(4): 1-44.
- [52] Zhong M, Chen R, Chen X, Fogarty J, Wobbrock J O. ScreenAudit: Detecting Screen Reader Accessibility Errors in Mobile Apps Using Large Language Models[C]//Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems. 2025: 1-19.
- [53] Usman A, Ibrahim R, Sulaiman M A, Anka S I. AN IN-DEPTH ANALYSIS OF MACHINE LEARNING BASED TECHNIQUES FOR AUTOMATED TESTING OF ANDROID APPLICATIONS[J]. International Journal of Communication Networks and Information Security, 2024, 16(3): 663-683.
- [54] Vendome C, Solano D, Liñán S, Linares-Vásquez M. Can everyone use my app? an empirical study on accessibility in android apps[C]//2019 IEEE International Conference on Software Maintenance and Evolution (ICSME). IEEE, 2019: 41-52.
- [55] Mateus D A, Aparecido da Silva F A C, Freire A P. Pandemic Crisis Brings More Digital Governmental Services to Mobile Devices- But Are They Accessible to People with Disabilities?[C]//Proceedings of the 39th ACM International Conference on Design of Communication. 2021: 197-204.
- [56] Eler M M, Rojas J M, Ge Y, Fraser G. Automated accessibility testing of mobile apps[C]//2018 IEEE 11th International Conference on Software Testing, Verification and Validation (ICST). IEEE, 2018: 116-126.
- [57] Serra L C, Carvalho L P, Ferreira L P, Vaz J B S, Freire A P. Accessibility evaluation of e-government mobile applications in Brazil[J]. Procedia Computer Science, 2015, 67: 348-357.
- [58] Lopes R, Façanha A R, Viana W. I can't pay! Accessibility analysis of mobile banking apps[C]//Proceedings of the Brazilian Symposium on Multimedia and the Web. 2022: 253-257.
- [59] Mateus D A, Silva C A, Eler M M, Freire A P. Accessibility of mobile applications: evaluation by users with visual impairment and by automated tools[C]//Proceedings of the 19th Brazilian Symposium on Human Factors in Computing Systems. 2020: 1-10.
- [60] Sun W, Fang C, Ge Y, Hu Y, Chen Y, Zhang Q, Ge X, Liu Y, Chen Z. "A survey of source code search: A 3-dimensional perspective." ACM Transactions on Software Engineering and Methodology 33.6 (2024): 1-51.
- [61] Keele, Staffs. Guidelines for performing systematic literature reviews in software engineering. Vol. 5. Technical report, ver. 2.3 ebse technical report. ebse, 2007.
- [62] Page M J, Moher D, Bossuyt P M, Boutron I, Hoffmann T C, Mulrow C D, Shamseer L, Tetzlaff J M, Akl E A, Brennan S E, Chou R, Glanville J, Grimshaw J M, Hróbjartsson A, Lalu M M, Li T, Loder E W, Mayo-Wilson E, McDonald S, McGuinness L A, Stewart L A, Thomas J, Tricco A C, Welch V A, Whiting P, McKenzie J E. "PRISMA 2020 explanation and elaboration: updated guidance and exemplars for reporting systematic reviews." bmj 372 (2021).
- [63] Wohlin C, Runeson P, Host M, Ohlsson M C, Regnell B, Wesslen A. Experimentation in software engineering. Vol. 236. Berlin: Springer, 2012.
- [64] Dybå, Tore, and Torgeir Dingsøy. "Empirical studies of agile software development: A systematic review." Information and software technology 50.9-10 (2008): 833-859.
- [65] Lloyd-Esenkaya T, Lloyd-Esenkaya V, O' Neill E, Proulx M J. Multisensory inclusive design with sensory substitution[J]. Cognitive Research: Principles and Implications, 2020, 5(1): 37.
- [66] Jordan J B, Van Hyning V, Jones M A, Bradley-Montgomery R L, Bottner E, Tansil E. Information Wayfinding of Screen Reader Users: Five Personas to Expand Conceptualizations of User Experiences[C]//Proceedings of the 26th International ACM SIGACCESS Conference on Computers and Accessibility. 2024: 1-7.
- [67] Kim H, Hwang H, Gwak S, Yoon J, Park K. Improving communication and promoting social inclusion for hearing-impaired users:

Usability evaluation and design recommendations for assistive mobile applications[J]. Plos one, 2024, 19(7): e0305726.

- [68] Huang M, McGrew G, Bodine C. Social Media Apps: A Paradigm for Examining Usability of Mobile Apps for Working Age Adults with Mild-Moderate Cognitive Disabilities[J]. ACM Transactions on Accessible Computing, 2025.
- [69] Alshayban A, Ahmed I, Malek S. Accessibility issues in android apps: state of affairs, sentiments, and ways forward[C]//Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering. 2020: 1323-1334.
- [70] Mehralian F, He Z, Malek S. Automated Accessibility Analysis of Dynamic Content Changes on Mobile Apps[C]//IEEE/ACM 47th International Conference on Software Engineering (ICSE). 2025: 1-13.
- [71] Oliveira A D A, Dos Santos P S H, Marcílio Júnior W E, Aljedaani W, Eler D M, Eler M M. Analyzing Accessibility Reviews Associated with Visual Disabilities or Eye Conditions[C]//Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems. 2023: 1-14.
- [72] Holanda Muniz J L, Mesquita Feijó Rabelo D, Viana W. Assessing Accessibility Levels in Mobile Applications Developed from Figma Templates[C]//Proceedings of the 17th International Conference on Pervasive Technologies Related to Assistive Environments. 2024: 316-321.
- [73] Fok R, Zhong M, Ross A S, Fogarty J, Wobbrock J O. A large-scale longitudinal analysis of missing label accessibility failures in android apps[C]//Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems. 2022: 1-16.
- [74] Di Gregorio M, Di Nucci D, Palomba F, Vitiello G. The making of accessible android applications: an empirical study on the state of the practice[J]. Empirical Software Engineering, 2022, 27(6): 145.
- [75] Mehralian F, Salehnamadi N, Malek S. Data-driven accessibility repair revisited: on the effectiveness of generating labels for icons in Android apps[C]//Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. 2021: 107-118.
- [76] Kashif S, Ahmad M, Shahid M, Habib M A, Rizwan K. Development of An Automated Accessibility Evaluation Plugin Tool for Mobile Applications[C]//2022 3rd International Conference on Innovations in Computer Science & Software Engineering (ICONICS). IEEE, 2022: 1-10.
- [77] Zhang X, De Greef L, Swearngin A, White S, Murray K, Yu L, Shan Q, Nichols J, Wu J, Fleizach C, Everitt A, Bigham J P. Screen recognition: Creating accessibility metadata for mobile applications from pixels[C]//Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems. 2021: 1-15.
- [78] He Z, Huq S F, Malek S. "I tend to view ads almost like a pestilence": On the Accessibility Implications of Mobile Ads for Blind Users[C]//Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. 2024: 1-13.
- [79] Acosta-Vargas P, Salvador-Ullauri L, Jadán-Guerrero J, Guevara C, Sanchez-Gordon S, Calle-Jimenez T, Lara P, Medina A, Nunes I L. Accessibility assessment in mobile applications for android[C]//Advances in Human Factors and Systems Interaction: Proceedings of the AHFE 2019 International Conference on Human Factors and Systems Interaction, July 24-28, 2019, Washington DC, USA 10. Springer International Publishing, 2020: 279-288.
- [80] Ross A S, Zhang X, Fogarty J, Wobbrock J O. An epidemiology-inspired large-scale analysis of android app accessibility[J]. ACM Transactions on Accessible Computing (TACCESS), 2020, 13(1): 1-36.
- [81] Akter T, Cha Y, Figueira I, Branham S M, Piper A M. "If I'm supposed to be the facilitator, I should be the host": understanding the accessibility of videoconferencing for blind and low vision meeting facilitators[C]//Proceedings of the 25th International ACM SIGACCESS Conference on Computers and Accessibility. 2023: 1-14.
- [82] Ma S, Chen C, Khalajzadeh H, John Grundy A first look at dark mode in real-world android apps[J]. ACM Transactions on Software Engineering and Methodology, 2023, 33(1): 1-26.
- [83] Garcia A C, de Lara S M A, Duarte L M C, Fortes R P M, Rodrigues K R H. Early accessibility testing-an automated kit for Android developers[C]//Proceedings of the 29th Brazilian Symposium on Multimedia and the Web. 2023: 11-15.
- [84] Dos Santos P S H, Oliveira A D A, De Jesus T B N, Aljedaani W, Eler M M. Evolution may come with a price: analyzing user reviews to understand the impact of updates on mobile apps accessibility[C]//Proceedings of the XXII Brazilian Symposium on Human Factors in Computing Systems. 2023: 1-11.
- [85] Tazi F, Saka S, Opp G, Neupane S, Das S, De Carli L, Ray I. Accessibility evaluation of IoT Android mobile companion APPs[C]//Extended Abstracts of the 2023 CHI Conference on Human Factors in Computing Systems. 2023: 1-7.
- [86] Chen M L, Hsieh K L, Hsieh S L, Hsieh M C, Lo D C T, Hsu J L. Barriers to Using Mobile App-Based Cognitive Testing in Older Adults with Probable Alzheimer's Disease: A Qualitative Study[J]. Brain Sciences, 2025, 15(5): 464.
- [87] Chen J, Swearngin A, Wu J, Barik T, Nichols J, Zhang X. Towards complete icon labeling in mobile applications[C]//Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems. 2022: 1-14.
- [88] Naranjo-Puentes S, Escobar-Velásquez C, Vendome C, Linares-Vásquez M. A preliminary study on accessibility of augmented reality features in mobile apps[C]//2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER). IEEE, 2022: 454-458.
- [89] Mehralian F, Salehnamadi N, Huq S F, Malek S. Too much accessibility is harmful! automated detection and analysis of overly accessible elements in mobile apps[C]//Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering. 2022: 1-13.
- [90] Gu M, Pei L, Zhou S, Shen M, Wu Y, Gao Z, Wang Z, Shan S, Jiang W, Li Y, Bu J. Towards an Inclusive Mobile Web: A Dataset and Framework for Focusability in UI Accessibility[C]//Proceedings of the ACM on Web Conference 2025. 2025: 5096-5107.
- [91] Tanwar S, Rao P V M. Inequality in User Experience: Can Mobile User Interfaces that Help Sighted Users Create Barriers for Visually Challenged People?[C]//International Conference on Computers Helping People with Special Needs. Cham: Springer Nature Switzerland, 2024: 19-30.

- [92] Kameswaran V, Hulikal Muralidhar S. Cash, Digital Payments and Accessibility: A Case Study from Metropolitan India[J]. Proceedings of the ACM on human-computer interaction, 2019, 3(CSCW): 1-23.
- [93] Branham S M, Wahid S, Byrne-Haber S, Mazrui J, Muncharaz C, Myhill C. The State of Digital Accessibility[J]. Communications of the ACM, 2025, 68(4): 38-41.
- [94] Naeem M, Ozuem W, Ward P. Understanding the accessibility of retail mobile banking during the COVID-19 pandemic[J]. International Journal of Retail & Distribution Management, 2022, 50(7): 860-879.
- [95] Oliveira A D A, dos Santos P S H, Aljedaani W, Eler M M. Exploring the Influence of Software Evolution on Mobile App Accessibility: Insights from User Reviews[J]. Journal of the Brazilian Computer Society, 2024, 30(1): 584-607.
- [96] Reyes Arias J E, Kurtzhall K, Pham D, Mkaouer M W, Elglaly Y N. Accessibility feedback in mobile application reviews: A dataset of reviews and accessibility guidelines[C]//CHI Conference on Human Factors in Computing Systems Extended Abstracts. 2022: 1-7.
- [97] TestDevLab. Accessibility Testing: Using Assistive Technologies to Detect Accessibility Issues. 2025. <https://www.testdevlab.com/blog/accessibility-testing-using-assistive-technologies>.
- [98] World Institute on Disability. Accountability Now: Enforcing Accessibility Standards In the Mobile App Economy. 2025. <https://wid.org/wp-content/uploads/2025/05/2025-Accountability-Now-Mobile-App-WP-3.pdf>.
- [99] ArcTouch, Fable. State of Mobile App Accessibility. 2025. <https://arctouch.com/state-of-mobile-app-accessibility>.
- [100] Aljedaani W, Mkaouer M W, Eler M M, Kessentini M. Empirical investigation of accessibility bug reports in mobile platforms: A chromium case study[C]//Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems. 2024: 1-17.
- [101] Shinohara K, Kawas S, Ko A J, Ladner R E. Who teaches accessibility? A survey of US computing faculty[C]//Proceedings of the 49th ACM Technical Symposium on Computer Science Education. 2018: 197-202.
- [102] Bhagat S, Joshi P, Agarwal A, Gupta S. Accessibility evaluation of major assistive mobile applications available for the visually impaired[J]. arXiv preprint arXiv:2407.17496, 2024.
- [103] Wickramathilaka S, Grundy J, Madampe K, Haggag O. Accessibility Recommendations for Designing Better Mobile Application User Interfaces for Seniors[J]. arXiv preprint arXiv:2504.12690, 2025.
- [104] Google-Lint. Improve your code with lint checks. 2011. <https://developer.android.com/studio/write/lint>.
- [105] Indika A, Lee C, Wang H, Lisoway J, Peruma A, Kazman R. Exploring Accessibility Trends and Challenges in Mobile App Development: A Study of Stack Overflow Questions[J]. arXiv preprint arXiv:2409.07945, 2024.
- [106] Google. Accessibility-Test-Framework-for-Android. 2014. <https://github.com/google/Accessibility-Test-Framework-for-Android>.
- [107] IBM. Mobile Accessibility Checker. 2015. <https://www.ibm.com/able/mobile-accessibility-checker>.
- [108] Google. Write automated tests with UI Automator. 2017. <https://developer.android.com/training/testing/other-components/ui-automator>.
- [109] Patil N, Bhole D, Shete P. Enhanced UI Automator Viewer with improved Android accessibility evaluation features[C]//2016 International Conference on Automatic Control and Dynamic Optimization Techniques (ICACDOT). IEEE, 2016: 977-983.
- [110] Salehnamadi N, He Z, Malek S. Assistive-technology aided manual accessibility testing in mobile apps, powered by record-and-replay[C]//Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems. 2023: 1-20.
- [111] Barboza, Alex Frederico Ramos, and Marcelo Medeiros Eler. "VisioAux: a crowdsourcing approach for accessibility evaluation in mobile devices." Simpósio Brasileiro sobre Fatores Humanos em Sistemas Computacionais (IHC). SBC, 2025.
- [112] Google. Espresso. 2013. <https://developer.android.com/training/testing/espresso>.
- [113] Roboelectric (Open Source Project). 2017. Roboelectric. <https://roboelectric.org/>.
- [114] Google. EarlGrey: iOS UI Automation Test Framework. 2016. <https://github.com/google/EarlGrey>.
- [115] Google. XCTest. 2013. <https://developer.apple.com/documentation/xctest>.
- [116] KIF. KIF: Keep it functional - an ios functional testing framework. 2016. <https://github.com/kif-framework/KIF>.
- [117] Li X, Chang N, Wang Y, Huang H, Pei Y, Wang L, Li X. ATOM: Automatic maintenance of GUI test scripts for evolving mobile applications[C]//2017 IEEE International Conference on Software Testing, Verification and Validation (ICST). IEEE, 2017: 161-171.
- [118] Kochhar P S, Thung F, Nagappan N, Zimmermann T, Lo D. Understanding the test automation culture of app developers[C]//2015 IEEE 8th International Conference on Software Testing, Verification and Validation (ICST). IEEE, 2015: 1-10.
- [119] Cruz L, Abreu R, Lo D. To the attention of mobile software developers: guess what, test your app! [J]. Empirical Software Engineering, 2019, 24: 2438-2468.
- [120] Pan M, Xu T, Pei Y, Li Z, Zhang T, Li X. Gui-guided test script repair for mobile apps[J]. IEEE Transactions on Software Engineering, 2020, 48(3): 910-929.
- [121] Ross A S, Zhang X, Fogarty J, Wobbrock J O. Examining image-based button labeling for accessibility in Android apps through large-scale analysis[C]//Proceedings of the 20th International ACM SIGACCESS Conference on Computers and Accessibility. 2018: 119-130.
- [122] Yan S, Ramachandran P G. The current status of accessibility in mobile apps[J]. ACM Transactions on Accessible Computing (TACCESS), 2019, 12(1): 1-31.

- [123] Hao S, Liu B, Nath S, Halfond W G J, Govindan R. Puma: Programmable ui-automation for large-scale dynamic analysis of mobile apps[C]//Proceedings of the 12th annual international conference on Mobile systems, applications, and services. 2014: 204-217.
- [124] Ravindranath L, Nath S, Padhye J, Balakrishnan H. Automatic and scalable fault detection for mobile applications[C]//Proceedings of the 12th annual international conference on Mobile systems, applications, and services. 2014: 190-203.
- [125] Google-Monkey.UI/Application Exerciser Monkey.2019.<https://developer.android.com/studio/test/other-testing-tools/monkey>.
- [126] Lin J W, Salehnamadi N, Malek S. Test automation in open-source android apps: A large-scale empirical study[C]//Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering. 2020: 1078-1089.
- [127] Alotaibi A S, Chiou P T, Halfond W G J. Automated detection of talkback interactive accessibility failures in android applications[C]//2022 IEEE Conference on Software Testing, Verification and Validation (ICST). IEEE, 2022: 232-243.
- [128] Salehnamadi N, Mehralian F, Malek S. Groundhog: An automated accessibility crawler for mobile apps. In 2022 37th IEEE[C]//ACM International Conference on Automated Software Engineering. IEEE, ACM New York, NY, USA, Rochester, Michigan, USA. 2022.
- [129] Li G, Baechler G, Tragut M, Li Y, Hartmann B. Learning to denoise raw mobile UI layouts for improving datasets at scale[C]//Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems. 2022: 1-13.
- [130] Stevens R, Gibler C, Crussell J, Erickson J, Chen H. Investigating user privacy in android ad libraries[C]//Workshop on Mobile Security Technologies (MoST). 2012, 10: 195-197.
- [131] Crussell J, Stevens R, Chen H. Madfraud: Investigating ad fraud in android applications[C]//Proceedings of the 12th annual international conference on Mobile systems, applications, and services. 2014: 123-134.
- [132] Alshayban A, Malek S. AccessiText: automated detection of text accessibility issues in Android apps[C]//Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. 2022: 984-995.
- [133] Su Y, Chen C, Wang J, Liu Z, Wang D, Li S, Wang Q. The metamorphosis: Automatic detection of scaling issues for mobile apps[C]//Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering. 2022: 1-12.
- [134] Liu Z, Chen C, Wang J, Huang Y, Hu J, Wang Q. Owl eyes: Spotting ui display issues via visual understanding[C]//Proceedings of the 35th IEEE/ACM international conference on automated software engineering. 2020: 398-409.
- [135] Liu Z, Chen C, Wang J, Huang Y, Hu J, Wang Q. Nighthawk: Fully automated localizing ui display issues via visual understanding[J]. IEEE Transactions on Software Engineering, 2022, 49(1): 403-418.
- [136] Ali A, Xia Y, Navid Q, Khan Z A, Khan J A, Aldakheel E A, Khafaga D. Mobile-UI-Repair: a deep learning based UI smell detection technique for mobile user interface[J]. PeerJ Computer Science, 2024, 10: e2028.
- [137] Liang X, Qi J, Gao Y, Peng C, Yang P. Ag3: Automated game gui text glitch detection based on computer vision[C]//Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. 2023: 1879-1890.
- [138] Zhang M, Liu H, Song S, Chen C, Huang P, Zhao J. Are your apps accessible? A GCN-based accessibility checker for low vision users[J]. Information and Software Technology, 2024, 174: 107518.
- [139] Taeb M, Swearngin A, Schoop E, Cheng R, Jiang Y, Nichols J. Axnav: Replaying accessibility tests from natural language[C]//Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems. 2024: 1-16.
- [140] Carvalho M C N, Dias F S, Reis A G S, Freire A P. Accessibility and usability problems encountered on websites and applications in mobile devices by blind and normal-vision users[C]//Proceedings of the 33rd Annual ACM symposium on applied computing. 2018: 2022-2029.
- [141] Alotaibi A S, Chiou P T, Halfond W G J. Automated repair of size-based inaccessibility issues in mobile applications[C]//2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE). IEEE, 2021: 730-742.
- [142] Xu Y, Li Z, Liu H, Liu Y. AGAA: An Android GUI Accessibility Adapter for Low Vision Users[C]//2023 IEEE 47th Annual Computers, Software, and Applications Conference (COMPSAC). IEEE, 2023: 412-421.
- [143] Alotaibi A S, Chiou P T, Tawsif F M, Halfond W G J. Scalefix: An automated repair of UI scaling accessibility issues in android applications[C]//2023 IEEE International Conference on Software Maintenance and Evolution (ICSME). IEEE, 2023: 147-159.
- [144] Linares-Vásquez M, Bavota G, Bernal-Cárdenas C, Di Penta M, Oliveto R, Poshyvanyk D. Multi-objective optimization of energy consumption of guis in android apps[J]. ACM Transactions on Software Engineering and Methodology (TOSEM), 2018, 27(3): 1-47.
- [145] Zhou Y, Chen C, Huang P, Wang Q. Don't Confuse! Redrawing GUI Navigation Flow in Mobile Apps for Visually Impaired Users[J]. IEEE Transactions on Software Engineering, 2024.
- [146] Chen J, Chen C, Xing Z, Xu X, Zhu L, Li G, Wang J. Unblind your apps: Predicting natural-language labels for mobile gui components by deep learning[C]//Proceedings of the ACM/IEEE 42nd international conference on software engineering. 2020: 322-334.
- [147] Gleason C, Carrington P, Cassidy C, Morris M R, Kitani K M, Bigham J P. "It's almost like they're trying to hide it" : How user-provided image descriptions have failed to make Twitter accessible[C]//The World Wide Web Conference. 2019: 549-559.
- [148] Zhang X, Ross A S, Fogarty J. Robust annotation of mobile application interfaces in methods for accessibility repair and enhancement[C]//Proceedings of the 31st Annual ACM Symposium on User Interface Software and Technology. 2018: 609-621.

- [149] Zhang X, Ross A S, Caspi A, Fogarty J, Wobbrock J O. Interaction proxies for runtime repair and enhancement of mobile application accessibility[C]//Proceedings of the 2017 CHI conference on human factors in computing systems. 2017: 6024-6037.
- [150] Li Y, Li G, He L, Zheng J, Li H, Guan Z. Widget captioning: Generating natural language description for mobile user interface elements[J]. arXiv preprint arXiv:2010.04295, 2020.
- [151] Ma Y, Yu C Y, Yan M. Icon Label Generation for Mobile Applications by Mean Teacher Learning[J]. 2022.
- [152] Ross A S, Zhang X, Fogarty J, Wobbrock J O. Epidemiology as a framework for large-scale mobile application accessibility assessment[C]//Proceedings of the 19th international ACM SIGACCESS conference on computers and accessibility. 2017: 2-11.
- [153] Haque S, Csallner C. Inferring Alt-text For UI Icons With Large Language Models During App Development[J]. arXiv preprint arXiv:2409.18060, 2024.
- [154] Haque S, Csallner C. Early Accessibility: Automating Alt-Text Generation for UI Icons During App Development[J]. arXiv preprint arXiv:2504.13069, 2025.
- [155] Lee K, Joshi M, Turc I R, Smith E M, FitzGerald N, Koura P S, Al-Rfou R, Ramachandran D. Pix2struct: Screenshot parsing as pretraining for visual language understanding[C]//International Conference on Machine Learning. PMLR, 2023: 18893-18912.
- [156] Beyer L, Steiner A, Pinto A S, Kolesnikov A, Wang X, Salz D, Neumann M, Alabdulmohsin I, Tschannen M, Bugliarello E, Unterthiner T, Keyzers D, Koppula S, Liu F, Grycner A, Gritsenko A, Houlsby N, Kumar M, Rong K, Eisenschlos J, Kabra R, Bauer M, Bošnjak M, Chen X, Minderer M, Voigtlaender P, Bica I, Balazevic I, Puigcerver J, Papalampidi P, Henaff O, Xiong X, Soricut R, Harmsen J, Zhai X. H Paligemma: A versatile 3b vlm for transfer[J]. arXiv preprint arXiv:2407.07726, 2024.
- [157] Liu Z, Chen C, Wang J, Chen M, Wu B, Huang Y, Hu J, Wang Q. Unblind text inputs: predicting hint-text of text input in mobile apps via LLM[C]//Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems. 2024: 1-20.
- [158] Mehralian F, Barik T, Nichols J, Swearngin A. Automated Code Fix Suggestions for Accessibility Issues in Mobile Apps[J]. arXiv preprint arXiv:2408.03827, 2024.
- [159] Xiang JH, Xu XY, Kong FC, Peng P, Zhang Z, Zhang YQ. Survey on Application and Development of Large Language Models in Software Defect Detection and Repair. Ruan Jian Xue Bao/Journal of Software, 2025, 36(4): 1489-1529 (in Chinese). <http://www.jos.org.cn/1000-9825/7268.htm>.
- [160] Hou X, Zhao Y, Liu Y, Yang Z, Wang K, Li L, Luo X, Lo D, Grundy J, Wang H. Large language models for software engineering: A systematic literature review[J]. ACM Transactions on Software Engineering and Methodology, 2024, 33(8): 1-79.
- [161] Yang B, Li S. UISGPT: Automated Mobile UI Design Smell Detection with Large Language Models[J]. Electronics, 2024, 13(16): 3127.
- [162] Chemnad K, Othman A. Digital accessibility in the era of artificial intelligence—Bibliometric analysis and systematic review[J]. Frontiers in Artificial intelligence, 2024, 7: 1349668.
- [163] Hamideh Kerdar S, Bächler L, Kirchhoff B M. The accessibility of digital technologies for people with visual impairment and blindness: a scoping review[J]. Discover Computing, 2024, 27(1): 24.
- [164] Alcocer J P S, Merino L, Fernandez-Blanco A, Ravelo-Mendez W, Escobar-Velasquez C, Linares-Vasquez M. A Developer's Guide to Building and Testing Accessible Mobile Apps[C]//2025 IEEE Conference on Software Testing, Verification and Validation (ICST). IEEE, 2025: 769-770.
- [165] Collins E, Neto A, Vincenzi A, Maldonado J C. Deep reinforcement learning based android application gui testing[C]//Proceedings of the XXXV Brazilian Symposium on Software Engineering. 2021: 186-194.
- [166] Murase K, Takada S. Applying Reinforcement Learning for Automated Testing of Mobile Application Focusing on State Definition, Reward, and Learning Method[C]//SEKE. 2023: 64-69.
- [167] Yu S, Ling Y, Fang C, Zhou Q, Zhao Y, Chen C, Zhu S, Chen Z. LLM-Guided Scenario-based GUI Testing[J]. arXiv preprint arXiv:2506.05079, 2025.
- [168] Zhang Y, Chen S, Xie X, She Z, Hu H, Fang C, Guo Y, Sui Y, Chen Z. Scenario-Driven and Context-Aware Automated Accessibility Testing for Android Apps[C]//2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE). IEEE Computer Society, 2025: 630-630.
- [169] Moser E V, Yli-Hukka Högbäck J. Incorporating Web Accessibility into a CI/CD Pipeline: A study of Open Source Evaluation Tools[J]. 2024.
- [170] Ghaleb T, Abduljalil O, Hassan S. CI/CD Configuration Practices in Open-Source Android Apps: An Empirical Study[J]. ACM Transactions on Software Engineering and Methodology, 2024.
- [171] Wang W, Grundy J, Khalajzadeh H, Madugalla A. Designing Adaptive User Interfaces for mHealth Applications Targeting Chronic Disease: A User-Centered Approach[J]. ACM Transactions on Software Engineering and Methodology, 2025.
- [172] Creed C, Al-Kalbani M, Theil A, Sarcar S, Williams I. Inclusive AR/VR: accessibility barriers for immersive technologies[J]. Universal Access in the Information Society, 2024, 23(1): 59-73.

APPENDIX OVERVIEW

This appendix provides the full reference support for the problem sets presented in Tables 5-8. Each problem is linked to its corresponding evidence source to ensure traceability, transparency, and verifiability without interrupting the readability of the main text. The appendix is organized into four sections (Appendix A-D), each corresponding to one problem category: Perceivability, Operability, Understandability, and Robustness.

Appendix A. Supporting References for Table 5

Problem ① supported by:[23][54][57][69][72][73][74][75][76][77][78][79][80]
Problem ② supported by:[23][55][57][71][76][81][82][83][84][85]
Problem ③ supported by:[55][69][79]
Problem ④ supported by:[82][84][86]
Problem ⑤ supported by:[71][73][76][87]
Problem ⑥ supported by:[88]
Problem ⑦ supported by:[70][84]
Problem ⑧ supported by:[69][72][77][79][85]
Problem ⑨ supported by:[84][89]
Problem ⑩ supported by:[23][82]
Problem ⑪ supported by:[23][55][69][74][79][80][84][85][90]
Problem ⑫ supported by:[23][69][72][80]

Appendix B. Supporting References for Table 6

Problem ① supported by:[23][57][69]
Problem ② supported by:[80][90]
Problem ③ supported by:[58][78][84][90][58][91]
Problem ④ supported by:[58][78][90]
Problem ⑤ supported by:[57][77][89]
Problem ⑥ supported by:[23][55][74][84][90]
Problem ⑦ supported by:[81][91]
Problem ⑧ supported by:[89]

Appendix C. Supporting References for Table 7: Mobile Understandability Problems

Problem ① supported by:[23][69][72][80]
Problem ② supported by:[23][69]
Problem ③ supported by:[57][58][66][69][77][78][90][91]
Problem ④ supported by:[91][92]
Problem ⑤ supported by:[23][77][93]
Problem ⑥ supported by:[78][91][94][95]
Problem ⑦ supported by:[84][96]

Appendix D. Supporting References for Table 8

Problem ① supported by:[82]
Problem ② supported by:[80][84][85]
Problem ③ supported by:[92][100]
Problem ④ supported by:[84][95]
Problem ⑤ supported by:[78]
Problem ⑥ supported by:[17][19][69][74][101]